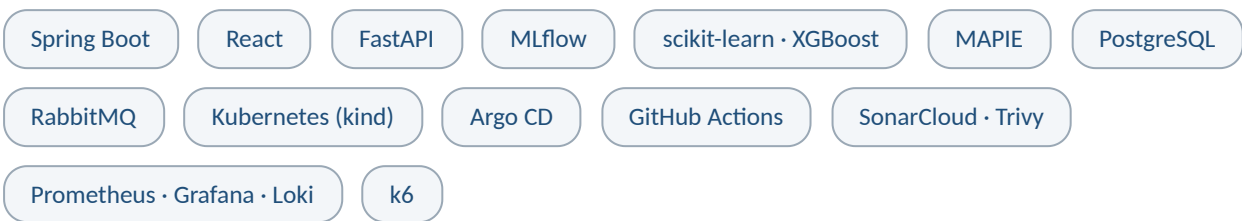


Plateforme FinTech MLOps haute disponibilité

Détection de fraude par carte bancaire : microservices, Kubernetes, benchmark
ML statistique, quantification d'incertitude et supervision intelligente



ÉQUIPE

Rôle	Responsabilité	Membre
P1	Architecte backend et chef de projet	[Nom Prénom]
P2	Data Scientist	[Nom Prénom]
P3	Ingénieur MLOps et serving	[Nom Prénom]
P4	Ingénieur DevOps	[Nom Prénom]
P5	Frontend et observabilité	[Nom Prénom]

ÉTABLISSEMENT / FILIÈRE

[Établissement] · LSI

ENCADRANT

[Nom de l'encadrant]

ANNÉE UNIVERSITAIRE

2026 – 2027

DATE DU DOCUMENT

30 septembre 2026

LIVRAISON DU PROJET

10 janvier 2027

DÉCISIONS ENREGISTRÉES

D1 à D32 (section 2.6)

Sommaire

1. Présentation du projet et objectifs	4
1.1 Contexte	4
1.2 Double objectif	4
1.3 Périmètre fonctionnel	4
1.4 Exigences non fonctionnelles (cibles proposées)	5
1.5 Contraintes	5
1.6 Hors périmètre	5
2. Vue d'ensemble de l'architecture	6
2.1 Les trois plans de la plateforme	6
2.2 Schéma global (exécution)	7
2.3 Schéma global (livraison)	8
2.4 Composants	8
2.5 Flux principal : évaluation d'une transaction	10
2.6 Décisions d'architecture	11
2.7 Traçabilité avec le sujet	12
3. Architecture microservices	14
3.1 Découpage en services	14
3.2 Vue des composants et des échanges	15
3.3 Cycle de vie d'une transaction	16
3.4 Détail des services	16
3.5 Communication asynchrone (RabbitMQ)	18
3.6 Communication synchrone et résilience	20
3.7 Flux de revue humaine	21
3.8 Sécurité	21
3.9 Données (PostgreSQL)	22
3.10 Conventions transverses	22
4. Architecture ML : données, benchmark, validation statistique et incertitude	24
4.1 Jeu de données : Sparkov	24
4.2 Découpage des données	24
4.3 Features et prétraitement	25
4.4 Gestion du déséquilibre des classes	25
4.5 Modèles comparés	26
4.6 Métriques	26
4.7 Protocole de benchmark	27
4.8 Validation statistique	28
4.9 Règle de sélection du modèle final	29
4.10 Calibration et quantification de l'incertitude	29
4.11 Visualisation des données (PCA, t-SNE, UMAP)	30
4.12 Reproductibilité	31
4.13 Livrables de la section	31
5. Architecture MLOps et serving du modèle	32
5.1 Cycle de vie du modèle	32
5.2 Serveur MLflow	34

5.3 Packaging du modèle	2
5.4 Portes de qualité avant promotion	2
5.5 Déploiement d'une nouvelle version du modèle (décision D20)	2
5.6 Où et quand le pipeline s'exécute	2
5.7 Service FastAPI (ml-service)	2
5.8 Métriques exposées par le ml-service	2
5.9 Suivi de la performance en production (décision D19)	2
5.10 Versionnement et rollback	2
5.11 Intégration avec risk-service	2
5.12 Livrables et calendrier	2
6. Architecture de déploiement : Docker et Kubernetes	2
6.1 Images Docker	2
6.2 Cluster local	2
6.3 Namespaces	2
6.4 Charges de travail	2
6.5 Ressources (requests et limits)	2
6.6 Autoscaling (HPA)	2
6.7 Sondes de santé (probes)	2
6.8 Mises à jour progressives et rollback	2
6.9 ConfigMaps et Secrets	2
6.10 Stockage	2
6.11 Ingress et équilibrage de charge	2
6.12 NetworkPolicies	2
6.13 Organisation des manifests	2
6.14 Livrables et calendrier	2
7. CI/CD et DevSecOps	2
7.1 Organisation du dépôt (monorepo)	2
7.2 Workflow Git	2
7.3 Vue d'ensemble de la chaîne	2
7.4 Pipeline d'une PR	2
7.5 Contrôles DevSecOps	2
7.6 Déploiement continu	2
7.7 Tests de bout en bout et rollback automatique	2
7.8 Workflows GitHub Actions	2
7.9 Limite documentée : un seul environnement	2
7.10 Livrables et calendrier	2
8. Observabilité : supervision, tableaux de bord et alertes	2
8.1 Architecture de supervision	2
8.2 Couverture des sources demandées par le sujet	2
8.3 Règles d'enregistrement	2
8.4 Tableaux de bord Grafana	2
8.5 Alertes	2
8.6 Logs centralisés (Loki, décision D27)	2
8.7 Accès et sécurité	2
8.8 Livrables et calendrier	2
9. Stratégie de tests de charge et de haute disponibilité	2

9.1 Hypothèses à vérifier	3
9.2 Outils	3
9.3 Modèle de charge	3
9.4 Calibration : test de capacité	3
9.5 Scénario principal : Normal → High → Overload → Recovery	3
9.6 Expériences de panne (chaos)	3
9.7 Définitions des mesures	3
9.8 Protocole d'exécution	3
9.9 Présentation des résultats	3
9.10 Limites documentées	3
9.11 Livrables et calendrier	3
10. Répartition des rôles et planning (version corrigée)	3
10.1 Rôles	3
10.2 Couverture des tâches du sujet	3
10.3 Charge de travail par période	3
10.4 Planning (S1 à S15)	3
10.5 Chaîne des livrables transmis	3
10.6 Jalons de contrôle	3
10.7 Organisation de l'équipe	3
10.8 Risques et mesures	3

1. Présentation du projet et objectifs

1.1 Contexte

Une institution financière traite en continu des paiements par carte. Une petite fraction de ces paiements est frauduleuse, et chaque fraude non détectée coûte de l'argent. À l'inverse, bloquer un paiement légitime dégrade l'expérience client.

La plateforme doit donc prendre une **décision de risque en temps réel** sur chaque transaction. Cette décision doit être **fiable, traçable et supervisée**. Lorsque le modèle n'est pas sûr de lui, la décision revient à un **analyste humain** au lieu de déclencher automatiquement une action à fort impact.

1.2 Double objectif

Le projet est évalué sur deux axes de poids égal.

Axe	Ce qu'il faut démontrer
Rigueur ML	Benchmark de 5 algorithmes, preuve statistique que le modèle retenu est significativement meilleur, quantification de l'incertitude de chaque prédiction
Qualité de production	Architecture microservices, déploiement Kubernetes scalable et auto-réparateur, CI/CD sécurisée, supervision complète avec alertes, résilience prouvée sous charge et en cas de panne

1.3 Périmètre fonctionnel

1. **Authentification** des utilisateurs, avec deux rôles :
 - **ADMIN** : administration de la plateforme,
 - **ANALYSTE** : revue humaine des transactions incertaines.
2. **Gestion des clients** (porteurs de carte).
3. **Réception des transactions**, envoyées par un **simulateur** qui rejoue le jeu de test Sparkov dans l'ordre chronologique. Le simulateur envoie aussi, avec un délai, la **vraie étiquette** de chaque transaction (rejet de paiement simulé). Cela permet de mesurer la performance réelle du modèle en production.
4. **Évaluation du risque** de chaque transaction par le modèle ML.
5. **Décision à trois issues** :
 - **APPROUVER** : le modèle est confiant que la transaction est légitime,
 - **REVUE HUMAINE** : le modèle est incertain,
 - **BLOQUER** : le modèle est confiant que la transaction est une fraude.
6. **File de revue humaine** : l'analyste consulte, valide ou rejette les transactions incertaines.
7. **Notifications en application** lors d'un blocage ou d'une mise en revue, enregistrées et affichées en temps réel dans le tableau de bord (pas d'email ni de SMS).
8. **Journal d'audit** complet et non modifiable de chaque décision (qui, quoi, quand, version du modèle).
9. **Tableau de bord React** : transactions, décisions, file de revue, visualisations des données (PCA, t-SNE, UMAP), performance et incertitude du modèle.

1.4 Exigences non fonctionnelles (cibles proposées)

Exigence	Cible	Mesurée par
Latence de bout en bout (p95), charge normale	< 500 ms	k6 + Prometheus
Latence de bout en bout (p95), charge élevée	< 2 s (seuil d'alerte du sujet)	k6 + Prometheus
Taux d'erreur, charge normale	< 1 %	k6 + Prometheus
Taux d'erreur, surcharge	< 5 % (seuil d'alerte du sujet)	k6 + Prometheus
Temps d'inférence ML (p95)	< 50 ms	Métrique FastAPI
Récupération après suppression d'un pod	< 60 s, sans interruption du service	Expérience de chaos
Délai de mise à l'échelle (HPA)	< 2 min après dépassement du seuil CPU	Métriques HPA
Traçabilité	100 % des décisions auditées avec la version du modèle	Service d'audit
Reproductibilité	Tout résultat ML reproductible depuis MLflow (seed, paramètres, données, code)	MLflow

Ces cibles sont des engagements mesurables. Les expériences de charge (section 9) les valident ou les invalident, et le README présente les résultats obtenus.

1.5 Contraintes

- **Équipe et délai** : 5 étudiants, 15 semaines, livraison le 10/01/2027 (dépôt visé le 09/01).
- **Infrastructure locale** : cluster Kubernetes local (kind) sur un portable de 16 Go de RAM, d'où des choix sobres en ressources.
- **Données** : jeu de données synthétique uniquement. Aucune donnée bancaire réelle n'est utilisée.
- **Démo** : 5 minutes maximum pour présenter l'ensemble.

1.6 Hors périmètre

- Paiements réels et intégration à un réseau de cartes.
- Déploiement cloud (possible ultérieurement, les manifests restant portables).
- Réentraînement automatique en continu. Le réentraînement est déclenché manuellement ou par la CI.

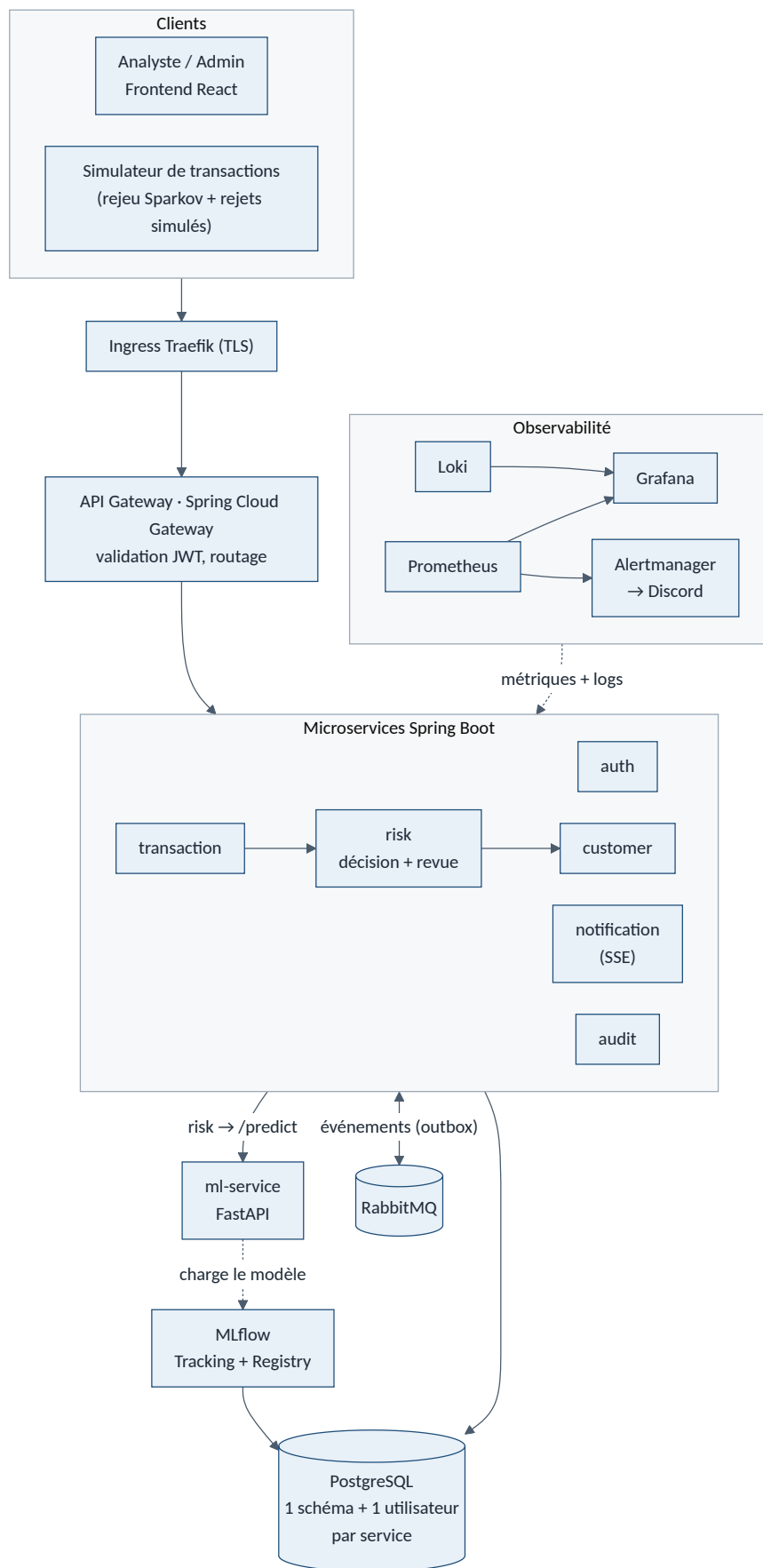
2. Vue d'ensemble de l'architecture

2.1 Les trois plans de la plateforme

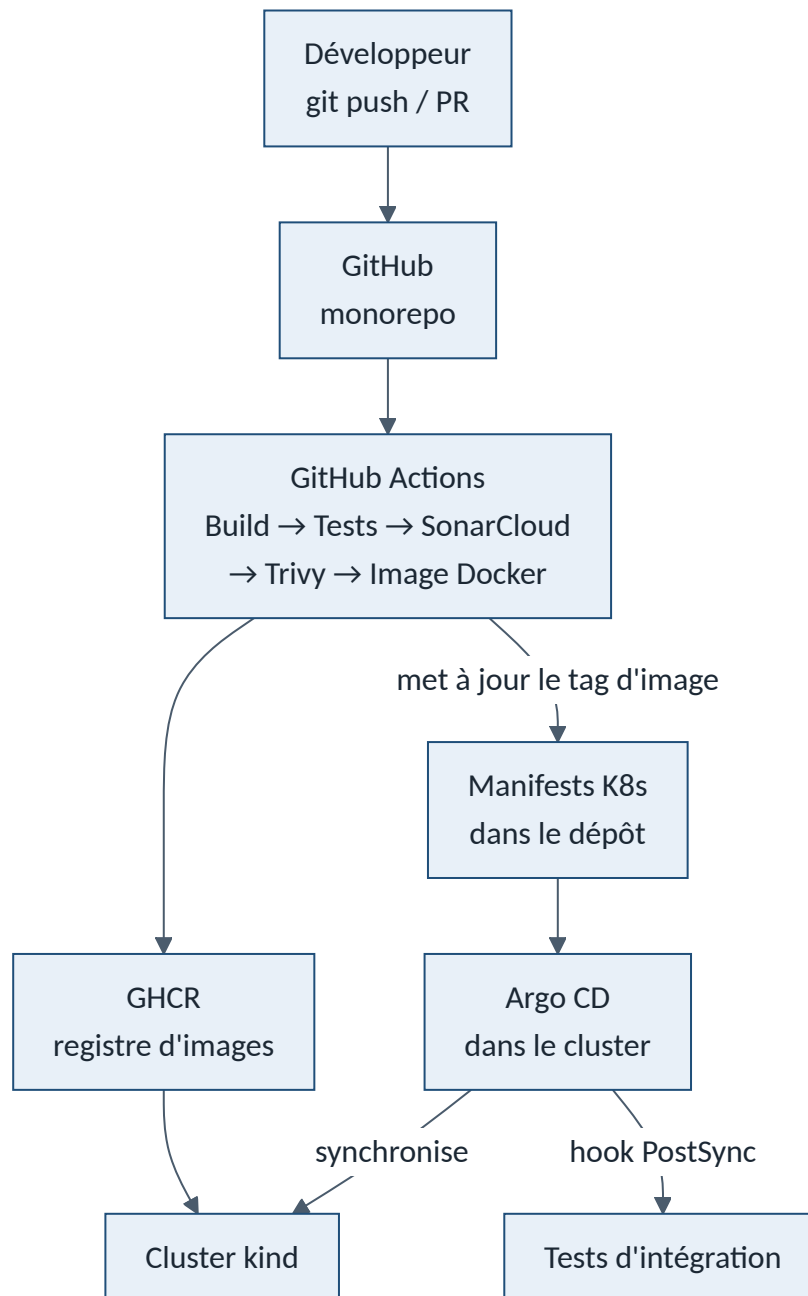
L'architecture s'organise en trois plans qui communiquent entre eux.

Plan	Rôle	Composants
Plan applicatif	Servir les utilisateurs et prendre les décisions	React, Ingress, API Gateway, 6 microservices Spring Boot, PostgreSQL, RabbitMQ
Plan ML	Entraîner, valider, versionner et servir le modèle	Pipeline d'entraînement, MLflow (tracking + registry), service FastAPI
Plan plateforme	Déployer, faire évoluer, superviser	Kubernetes (HPA, probes), GitHub Actions, Argo CD, Prometheus, Grafana, Alertmanager

2.2 Schéma global (exécution)



2.3 Schéma global (livraison)



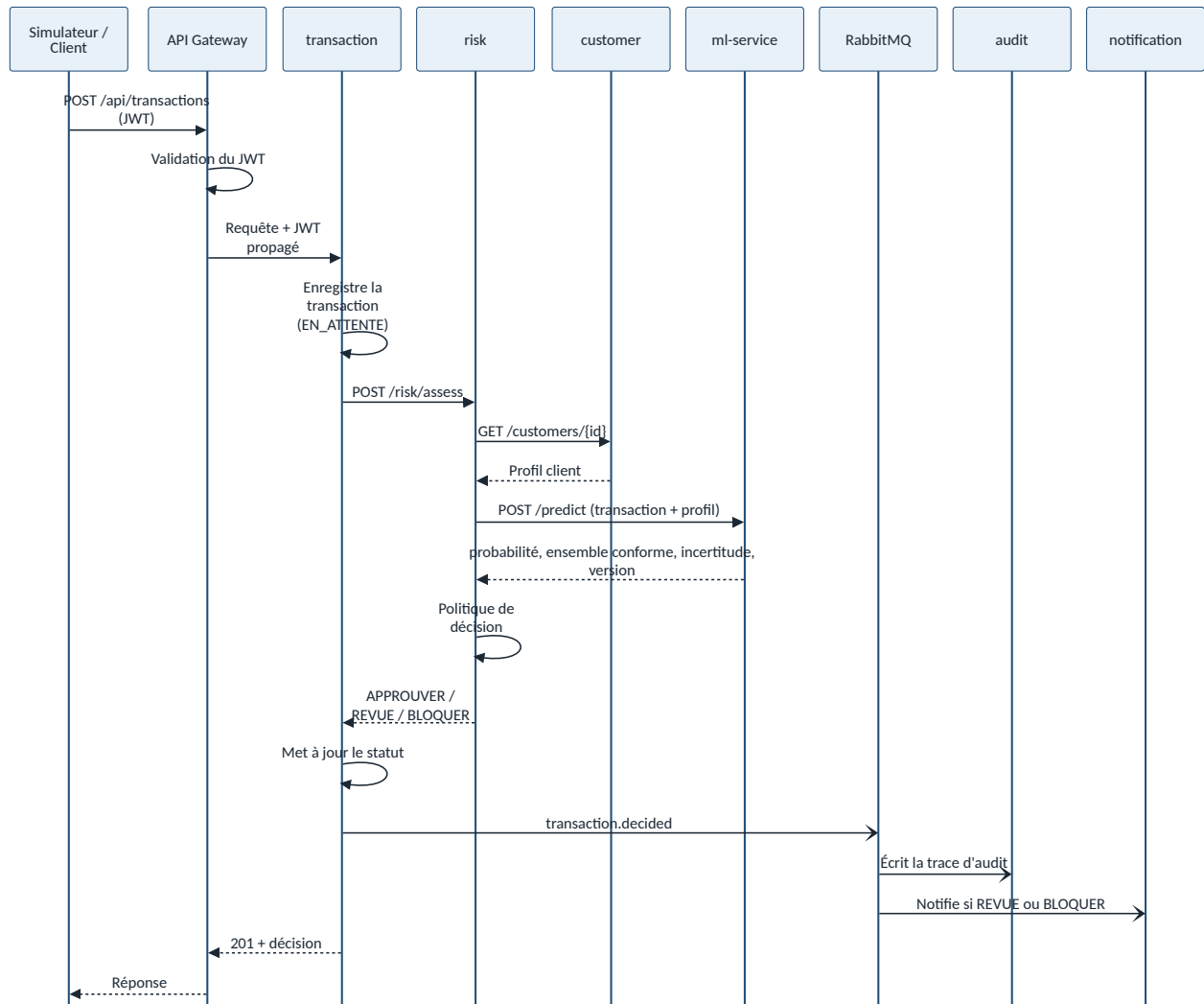
Argo CD **tire** les changements depuis Git. Le cluster local n'a donc pas besoin d'être accessible depuis Internet. Un rollback se fait par un `git revert` ou depuis l'interface d'Argo CD.

2.4 Composants

Composant	Technologie	Rôle	Responsable
frontend	React	Interface analyste/admin, visualisations, file de revue	P5
api-gateway	Spring Cloud Gateway	Point d'entrée unique, routage, validation JWT, limitation de débit	P1
auth-service	Spring Boot + Spring Security	Connexion, émission des JWT, gestion des rôles	P1

Composant	Technologie	Rôle	Responsable
customer-service	Spring Boot	Données des porteurs de carte	P1
transaction-service	Spring Boot	Réception et cycle de vie des transactions	P1
risk-service	Spring Boot + Resilience4j	Appel au modèle, politique de décision, file de revue humaine	P1
notification-service	Spring Boot	Notifications en application, diffusion temps réel (SSE) vers React	P1
audit-service	Spring Boot	Journal d'audit en ajout seul	P1
ml-service	FastAPI	Prédiction, probabilité, incertitude, version du modèle	P3
mlflow	MLflow	Suivi des expériences, artefacts, Model Registry	P3
postgres	PostgreSQL	Une instance, un schéma et un utilisateur par service	P1 (schémas), P4 (déploiement)
rabbitmq	RabbitMQ	Événements asynchrones vers Notification, Audit et Transaction	P1 (topologie), P4 (déploiement)
prometheus, grafana, alertmanager	kube-prometheus-stack	Collecte des métriques, tableaux de bord, alertes	P5
argocd	Argo CD	Déploiement GitOps	P4

2.5 Flux principal : évaluation d'une transaction



Politique de décision. Elle repose sur la prédiction conforme (MAPIE) appliquée à un modèle calibré. Pour chaque transaction, le modèle renvoie un **ensemble de prédiction** qui contient la vraie classe avec une probabilité garantie de $1 - \alpha$.

Ensemble conforme	Interprétation	Décision
{légitime}	Confiant : transaction légitime	APPROUVER
{fraude}	Confiant : transaction frauduleuse	BLOQUER
{légitime, fraude} ou ensemble vide	Incertain	REVUE HUMAINE

La valeur de α est fixée par P2 à partir des résultats expérimentaux (section 4).

Dégradation contrôlée. Si le ml-service est indisponible ou dépasse son délai, le circuit breaker (Resilience4j) s'ouvre. La transaction part alors en **REVUE HUMAINE**. Elle n'est jamais approuvée automatiquement sans avis du modèle. Ce choix « fail-safe » est adapté à la finance.

Synchrone ou asynchrone. La décision est **synchrone**, car le client attend une réponse. La notification et l'audit sont **asynchrones** via RabbitMQ : une panne de ces services ne bloque pas les paiements, et les messages sont traités à leur redémarrage.

Fiabilité des événements (outbox transactionnel). Un service n'envoie pas directement ses événements à RabbitMQ. Il les écrit dans une table `outbox_events` dans la même transaction que ses données métier. Un relais les publie ensuite. Même si un pod est tué entre l'enregistrement et la publication, aucun événement n'est perdu (détail en section 3.5).

2.6 Décisions d'architecture

#	Décision	Alternatives écartées	Justification
D1	Jeu de données Sparkov (fraude carte synthétique)	ULB, PaySim, IEEE-CIS	Contient clients, marchands et catégories, ce qui correspond directement aux services Customer et Transaction. Scores non triviaux, donc des tests statistiques pertinents. Découpage temporel déjà fourni.
D2	Monorepo GitHub	Un dépôt par service	Plus simple pour 5 personnes et pour Argo CD. Une seule PR peut modifier un contrat et ses deux côtés.
D3	Cluster local kind	Cloud managé	Gratuit et suffisant pour démontrer replicas, HPA, probes et rollback. Manifests portables vers le cloud.
D4	REST synchrone + RabbitMQ pour Notification et Audit	REST seul, Kafka	Découple les services non critiques du chemin de paiement. Plus léger que Kafka sur 16 Go.
D5	Une instance PostgreSQL , un schéma et un utilisateur par service	Une base par service	Isolation logique conservée (droits limités au schéma, migrations Flyway séparées) pour environ 1 Go de RAM économisé.
D6	JWT validé à la Gateway et propagé aux services	Jetons service-à-service, mTLS	Suffisant pour le périmètre. Chaque service revalide la signature (défense en profondeur).
D7	Découverte de services native Kubernetes	Eureka	Le DNS et les Services Kubernetes suffisent. Un composant de moins.
D8	GitHub Actions (CI) + Argo CD (CD GitOps)	Runner auto-hébergé, déploiement manuel	Le cluster local n'a pas à être exposé. Rollback par Git.
D9	Pipeline ML orchestré par GitHub Actions + MLflow	Prefect	Un outil de moins à apprendre et à déployer.
D10	SonarCloud	SonarQube auto-hébergé	Gratuit pour les dépôts publics et ne consomme pas la RAM du portable.
D11	Calibration + prédiction conforme (MAPIE)	Ensembles, bootstrap	Garantie statistique de couverture, ce qui rend la décision de revue humaine justifiable.
D12	Décision à trois issues	Deux issues (sans blocage auto)	Reproduit la pratique réelle : les cas certains sont automatisés, les cas incertains sont délégués à l'humain.
D13	Prétraitement dans le pipeline du modèle : le ml-service reçoit les champs bruts	Calcul des features dans risk-service (Java)	Le même code transforme les données à l'entraînement et en production. Aucun écart entraînement/production possible.
D14	Notifications en application uniquement (stockées + SSE vers React)	Email via MailHog, logs seuls	Léger, visible pendant la démo, sans composant supplémentaire.
D15	Outbox transactionnel pour la publication des événements	Publication après commit	Aucune perte d'événement lors des expériences de suppression de pods.
D16	Services Notification et Audit confiés à P1	Confiés à P4	Choix de l'équipe : tout le code Spring Boot est sous une même responsabilité, ce qui garantit la cohérence des conventions.

#	Décision	Alternatives écartées	Justification
D17	Pas de features d'historique client : modèle sans état	Agrégats sur 24 h calculés par transaction-service	Plus simple, sans stockage d'historique. Sparkov est déjà bien séparable avec les features propres à chaque transaction.
D18	Genre et âge conservés comme features, avec un audit d'équité	Exclure le genre, exclure les deux	Données synthétiques, cas de détection de fraude. L'audit par groupe rend le choix transparent.
D19	Rejets de paiement simulés : le simulateur envoie les vraies étiquettes avec un délai	Étiquettes issues de la revue humaine uniquement	Mesure réelle de la précision, du rappel et de la couverture conforme en production, y compris sur les décisions automatiques.
D20	Version du modèle épinglée dans Git , promue par PR fusionnée automatiquement si la CI est verte (paramètre <code>AUTO_MERGE</code>)	PR fusionnée manuellement, alias MLflow + rechargement automatique	Automatique tout en gardant Git comme source de vérité unique, le rollback Kubernetes et une trace d'audit complète.
D21	Contrôleur Ingress Traefik avec ressources Ingress standard	Ingress NGINX (retiré en mars 2026), Gateway API + Envoy Gateway	Maintenu, léger, et conforme à l'exigence « Ingress » du sujet.
D22	Sealed Secrets : secrets chiffrés committés dans Git	Secrets créés par script hors Git	Tout est déployé depuis Git par Argo CD, sans jamais exposer un secret en clair.
D23	Cluster kind à 3 nœuds (1 control-plane + 2 workers)	Nœud unique	Répartition des replicas sur deux nœuds et test de panne d'un nœud complet.
D24	Rollback automatique par Git : tests de bout en bout après déploiement, revert en cas d'échec	Argo Rollouts (canary + analyse Prometheus)	Simple, cohérent avec D20, sans outil ni RAM supplémentaires.
D25	Bonus de sécurité reportés (cosign, OWASP ZAP)	Les planifier dès maintenant	Ajoutés seulement s'il reste du temps après S12. Priorité aux exigences du sujet.
D26	Alertes envoyées sur Discord (webhook)	Slack, notifications en application, Grafana seul	Gratuit, reçu sur téléphone, très visible pendant la démo.
D27	Logs centralisés avec Loki + Grafana Alloy	Logs via kubectl uniquement	Recherche par <code>correlationId</code> et passage direct d'une métrique aux logs. Coût : ~0,6 Go de RAM.
D28	k6 exécuté depuis un second portable du même réseau local	k6 sur la machine du cluster	Mesures non faussées par le partage du processeur, et RAM libérée sur la machine du cluster.
D29	3 exécutions par expérience , médiane et étendue	Une seule exécution	Résultats robustes, dans le même esprit que la validation statistique de la partie ML.
D30	Spring Boot 4.1 + Java 21	Spring Boot 3.5, Java 25	La branche 3.x ne reçoit plus de correctifs open source depuis juin 2026. Java 21 est une version LTS familière à l'équipe.
D31	Spring Cloud Gateway réactif (WebFlux)	Spring Cloud Gateway Server MVC	La variante la plus documentée, adaptée au flux SSE, configurée surtout en YAML.
D32	Contrats OpenAPI écrits d'abord (contract-first)	Contrats générés depuis le code	Débloque P3, P4 et P5 dès S2. La CI vérifie que le code respecte le contrat.

2.7 Traçabilité avec le sujet

Tâche du sujet	Où dans l'architecture	Section
T1 Microservices	Gateway, 6 services, RabbitMQ, PostgreSQL	3
T2 Benchmark ML	Pipeline d'entraînement, MLflow	4
T3 Validation statistique	Étape de validation du pipeline	4

Tâche du sujet	Où dans l'architecture	Section
T4 Incertitude	MAPIE, politique de décision du risk-service, file de revue	4, 2.5
T5 MLflow & MLOps	Tracking, Registry, pipeline reproductible	5
T6 Serving FastAPI	ml-service, intégration avec risk-service	5
T7 Visualisation	Étape du pipeline, pages React	4, 5
T8 Docker & Kubernetes	Déploiements, Ingress, HPA, probes	6
T9 Charge & haute disponibilité	k6, expériences de chaos	9
T10 Monitoring	Prometheus, Grafana, Alertmanager	8
T11 CI/CD & DevSecOps	GitHub Actions, SonarCloud, Trivy, Argo CD	7

3. Architecture microservices

3.1 Découpage en services

Chaque service possède **un seul domaine métier** et **ses propres données**. Aucun service ne lit directement les tables d'un autre : l'accès aux données d'un autre domaine passe par son API REST ou par ses événements.

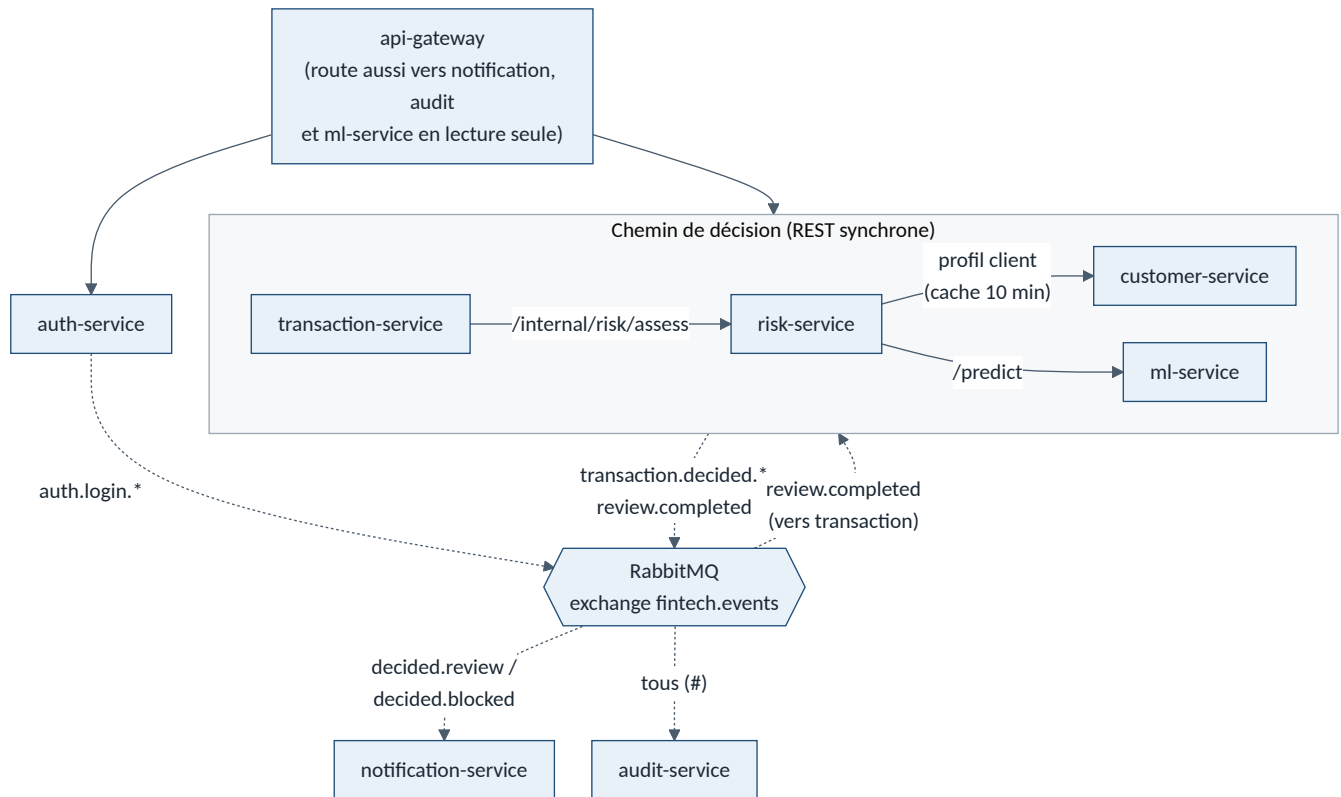
Service	Responsabilité	Schéma PostgreSQL	Port	Publie	Consomme
api-gateway	Point d'entrée unique, routage, validation JWT, CORS, identifiant de corrélation	aucun	8080	-	-
auth-service	Utilisateurs, rôles, émission et rafraîchissement des JWT, clé publique (JWKS)	auth	8081	auth.login.*	-
customer-service	Porteurs de carte (profil, adresse, localisation)	customer	8082	-	-
transaction-service	Réception des transactions et cycle de vie de leur statut	transaction	8083	transaction.decided.*	review.completed
risk-service	Appel au modèle, politique de décision, file de revue humaine	risk	8084	review.completed	-
notification-service	Notifications en application, diffusion en temps réel (SSE)	notification	8085	-	transaction.decided.review , transaction.decided.blocked
audit-service	Journal d'audit en ajout seul	audit	8086	-	tous les événements
ml-service (FastAPI)	Prédiction, probabilité, ensemble conforme, incertitude, version	aucun	8000	-	-

Socle technique commun (décision D30) : Java 21, **Spring Boot 4.1**, Spring Cloud Gateway **réactif (WebFlux)** pour la Gateway (D31), Spring MVC pour les autres services, Spring Security 7 (resource server), Spring Data JPA, Flyway, Spring AMQP, Resilience4j, Micrometer et springdoc-openapi.

Pourquoi Spring Boot 4.1. Toutes les versions 3.x ne reçoivent plus de correctifs open source depuis juin 2026, et la 4.0 n'en reçoit que jusqu'à fin décembre 2026. Avec une version non maintenue, l'analyse Trivy de la CI (section 7.5) signifierait des vulnérabilités impossibles à corriger.

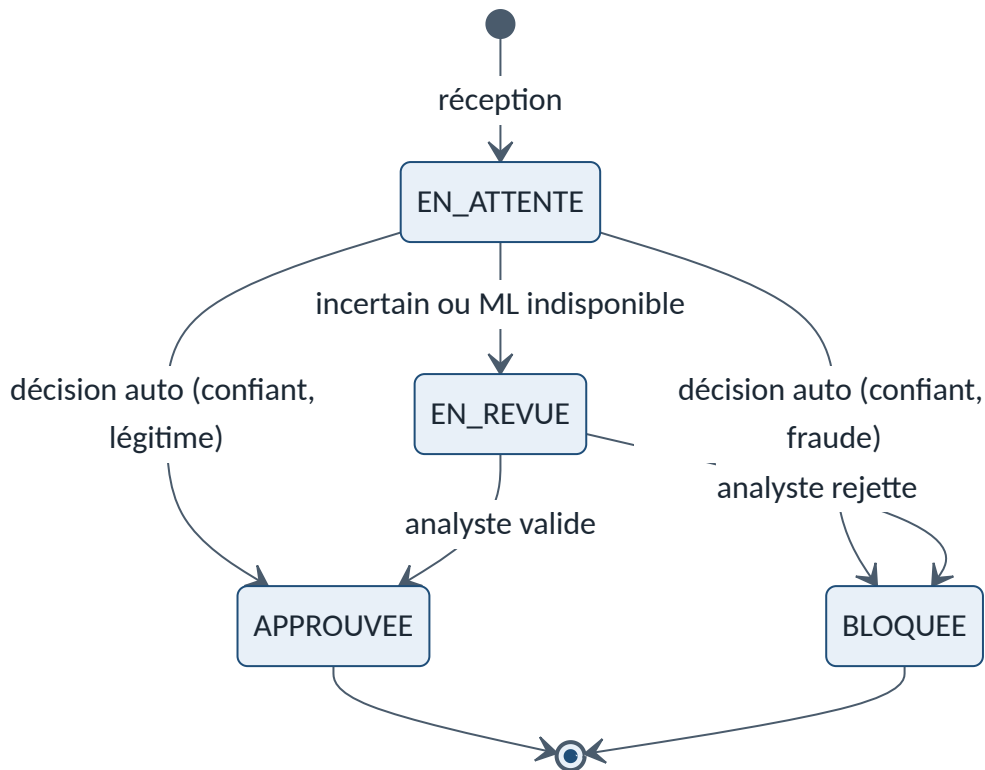
Bibliothèque partagée. Un module Maven **common** regroupe ce qui est identique dans les 7 services : identifiant de corrélation, format d'erreur, configuration de sécurité, enveloppe des événements, outbox, consommateur idempotent, configuration RabbitMQ et étiquettes de métriques.

3.2 Vue des composants et des échanges



Flèches pleines : appels REST synchrones. Flèches pointillées : événements RabbitMQ.

3.3 Cycle de vie d'une transaction



Chaque transaction enregistre aussi la **source de la décision** : **AUTO** (le modèle) ou **MANUELLE** (un analyste).

3.4 Détail des services

api-gateway

- **Routage.** Préfixe `/api/v1/{domaine}/**` vers le service correspondant. Les chemins `/internal/**` ne sont **jamais** routés : ils restent accessibles uniquement à l'intérieur du cluster.
- **Sécurité.** Validation de la signature et de l'expiration du JWT à partir des clés publiques d'auth-service (JWKS). Le jeton est ensuite propagé au service cible.
- **Corrélation.** Génère un en-tête `X-Correlation-Id` s'il est absent. Cet identifiant est propagé dans tous les appels, les logs et les événements, ce qui permet de suivre une transaction de bout en bout.
- **Limitation de débit.** Uniquement sur `/api/v1/auth/login` (protection contre la force brute), en mémoire, sans Redis. Elle n'est **pas appliquée** aux transactions, pour ne pas fausser les tests de charge.
- **Routage WebSocket/SSE** vers notification-service pour le flux temps réel.

auth-service

- **Endpoints :**
 - `POST /api/v1/auth/login`
 - `POST /api/v1/auth/refresh`
 - `GET /api/v1/auth/me`
 - CRUD des utilisateurs (ADMIN)
 - `GET /.well-known/jwks.json`
- **Jetons.** JWT signés en **RS256** : seul auth-service détient la clé privée, et les autres services vérifient avec la clé publique, sans secret partagé. Jeton d'accès de 15 min, jeton de rafraîchissement de 7 jours.

- **Rôles :**
 - `ADMIN` ,
 - `ANALYSTE` ,
 - `SIMULATEUR` : compte technique utilisé par le simulateur de transactions et par les tests de charge.
- **Données.** Mots de passe hachés avec **BCrypt**. Tables `users` et `refresh_tokens` .
- **Événements.** Publie `auth.login.succeeded` et `auth.login.failed` pour l'audit.

customer-service

- **Endpoints :**
 - `GET /api/v1/customers` (paginé, avec filtres)
 - `GET /api/v1/customers/{id}`
 - `POST /api/v1/customers`
 - `PUT /api/v1/customers/{id}`
- **Données.** Champs issus de Sparkov : nom, genre, date de naissance, profession, adresse, ville, population de la ville, latitude et longitude.
- **Numéro de carte.** Il n'est **jamais stocké en clair**. On conserve une empreinte HMAC-SHA256, qui sert d'identifiant de carte, et les 4 derniers chiffres pour l'affichage. C'est une bonne pratique inspirée de PCI-DSS.
- **Initialisation.** Les clients uniques du jeu Sparkov sont chargés par une migration de données au démarrage.

transaction-service

- **Endpoints :**
 - `POST /api/v1/transactions`
 - `GET /api/v1/transactions` (filtres : statut, client, période, montant)
 - `GET /api/v1/transactions/{id}`
- **Idempotence.** Le `trans_num` de Sparkov sert de clé d'idempotence. Si une transaction est renvoyée (par exemple lors d'un retry), elle n'est pas créée deux fois : la réponse initiale est renvoyée.
- **Données.** Client, marchand, catégorie, montant, date et heure, localisation du marchand, statut, source de la décision, dates de création et de mise à jour.
- **Événements publiés.** Après chaque décision, `transaction.decided.approved` , `transaction.decided.review` ou `transaction.decided.blocked` .
- **Événements consommés.** `review.completed` , qui fait passer la transaction de `EN_REVUE` à `APPROUEE` ou `BLOQUEE` (source `MANUELLE`).

risk-service

- **Endpoints :**
 - `POST /internal/risk/assess` (interne uniquement)
 - `GET /api/v1/risk/assessments/{transactionId}`
 - `GET /api/v1/risk/reviews?status=OUVERTE` (ANALYSTE)
 - `POST /api/v1/risk/reviews/{id}/decision` (ANALYSTE)
 - `GET /api/v1/risk/stats` (taux d'approbation, de revue et de blocage)
 - `POST /api/v1/risk/feedback` (SIMULATEUR) : vraie étiquette reçue en différé (rejet de paiement simulé)
 - `GET /internal/risk/labels`
- **Table `assessments`** . Probabilité, ensemble conforme, score d'incertitude, nom et version du modèle, décision, latence de l'appel ML, raison.

- **Table `reviews`** . Statut, analyste, décision finale, commentaire, dates d'ouverture et de clôture.
- **Concurrence**. Un **verrou optimiste** (`@Version`) empêche deux analystes de traiter la même revue en même temps. Le second reçoit une erreur `409 Conflict` .
- **Cache**. Le profil client est mis en cache localement (Caffeine, 10 min), ce qui évite un appel à customer-service pour chaque transaction.
- **Boucle de retour**. Deux sources de vraies étiquettes en production :
 - les **décisions des analystes**, sur les transactions revues ;
 - les **rejets de paiement simulés** envoyés par le simulateur, sur **toutes** les transactions, y compris les décisions automatiques.

risk-service rapproche chaque étiquette de la décision prise. Il expose des compteurs Prometheus (matrice de confusion, couverture conforme) et `/internal/risk/labels` pour le pipeline ML (section 5.9).

notification-service

- **Endpoints** :
 - `GET /api/v1/notifications`
 - `PATCH /api/v1/notifications/{id}/read`
 - `GET /api/v1/notifications/stream` (flux **SSE**, temps réel)
- **Types de notification**. `BLOPAGE` et `REVUE` , chacune liée à sa transaction.
- **Plusieurs replicas**. Le navigateur est connecté en SSE à **un seul** replica. Pour que chaque navigateur reçoive toutes les notifications :
 1. La queue durable `notification.alerts` est partagée entre les replicas : chaque message est enregistré une seule fois.
 2. Après l'enregistrement, le service publie sur un exchange `fanout` `notifications.live` .
 3. Chaque replica y possède sa propre queue temporaire et pousse le message à ses clients SSE.

audit-service

- **Endpoints** : `GET /api/v1/audit` (ADMIN), avec filtres par type, acteur, identifiant et période.
- **Table `audit_events`** . Identifiant de l'événement (unique), type, entité concernée, acteur, version du modèle, charge utile (JSONB), date de l'événement, date de réception.
- **Ajout seul, garanti par la base**. L'utilisateur PostgreSQL `audit_user` n'a que les droits `INSERT` et `SELECT` . Même un bug applicatif ne peut pas modifier ni supprimer une trace.

3.5 Communication asynchrone (RabbitMQ)

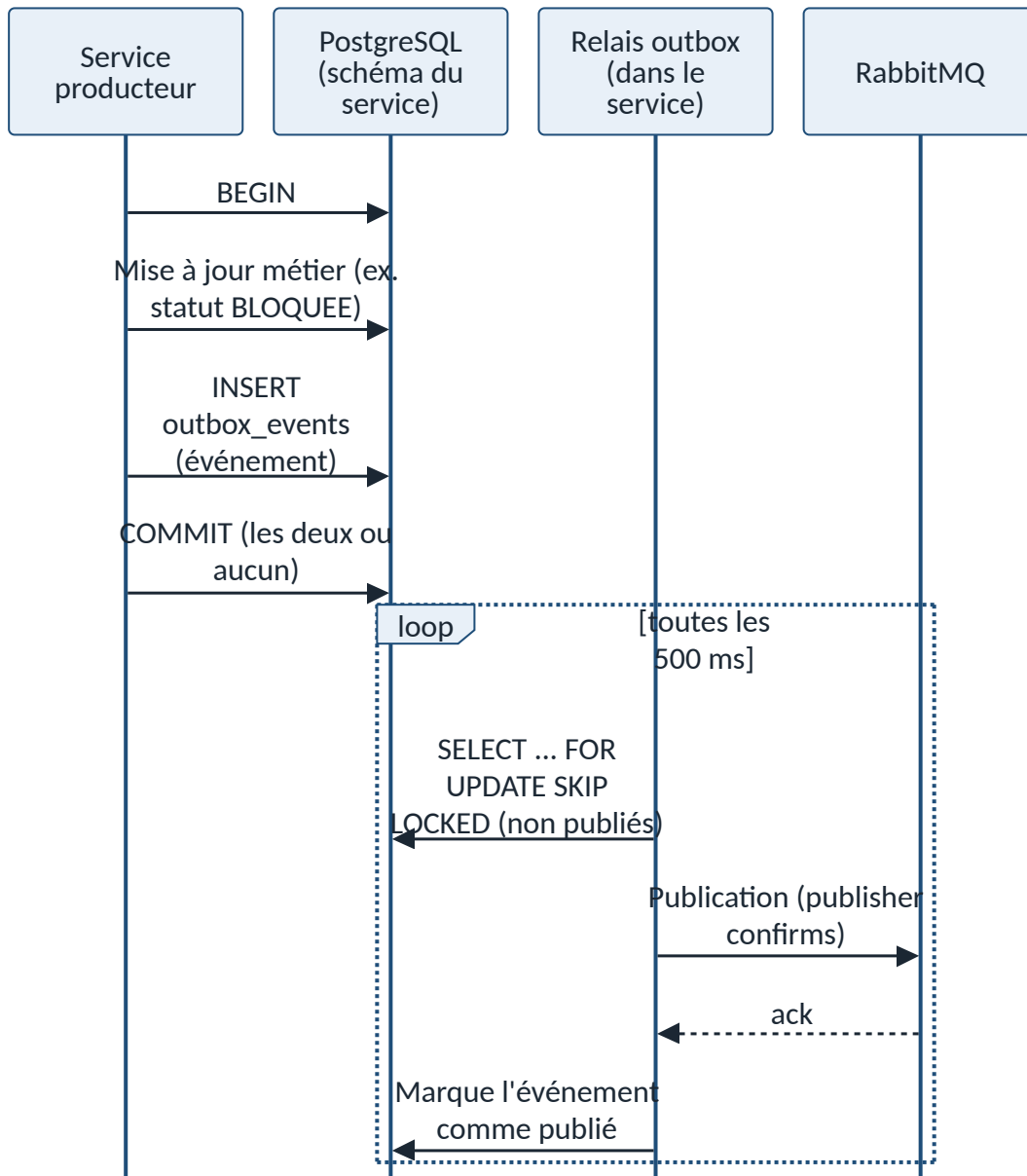
Topologie. Un exchange `topic` principal, `fintech.events` , et un exchange `fanout` , `notifications.live` , pour la diffusion temps réel.

Clé de routage	Producteur	Consommateurs (queue)
<code>transaction.decided.approved</code>	transaction-service	audit (<code>audit.all</code>)
<code>transaction.decided.review</code>	transaction-service	audit, notification (<code>notification.alerts</code>)
<code>transaction.decided.blocked</code>	transaction-service	audit, notification
<code>review.completed</code>	risk-service	audit, transaction (<code>transaction.review-results</code>)
<code>auth.login.succeeded</code> / <code>.failed</code>	auth-service	audit

Enveloppe commune des événements :

```
{
  "eventId": "uuid",
  "type": "transaction.decided.blocked",
  "version": 1,
  "occurredAt": "2026-11-20T14:03:12Z",
  "correlationId": "uuid",
  "producer": "transaction-service",
  "payload": { "transactionId": "...", "decision": "BLOQUEE", "modelVersion": "fraud-xgb:4" }
}
```

Outbox transactionnel (côté producteurs : auth, transaction, risk).



- **Atomicité.** La donnée métier et l'événement sont écrits dans la **même transaction**. Soit les deux existent, soit aucun.
- **Plusieurs replicas.** Si un pod meurt après le commit, un autre replica, ou le même après redémarrage, publie l'événement. **SKIP LOCKED** empêche deux replicas de publier le même événement en même temps.
- **Doublons.** En cas de crash entre la publication et le marquage, l'événement peut être publié deux fois. Les consommateurs idempotents l'ignorent.

- **Nettoyage.** Les événements publiés depuis plus de 7 jours sont purgés par un job planifié.
- **Supervision.** Le nombre d'événements en attente (`outbox_pending`) est exposé à Prometheus. S'il monte, c'est que RabbitMQ est en difficulté : une alerte se déclenche.

Garanties de livraison :

- **Au moins une fois.** Les messages et les queues sont durables, et l'acquittement est manuel après traitement.
- **Consommateurs idempotents.** Un message déjà traité (même `eventId`) est ignoré.
- **Échecs.** Après 3 tentatives, le message part dans une **dead-letter queue** (DLQ) par queue, supervisée dans Grafana.

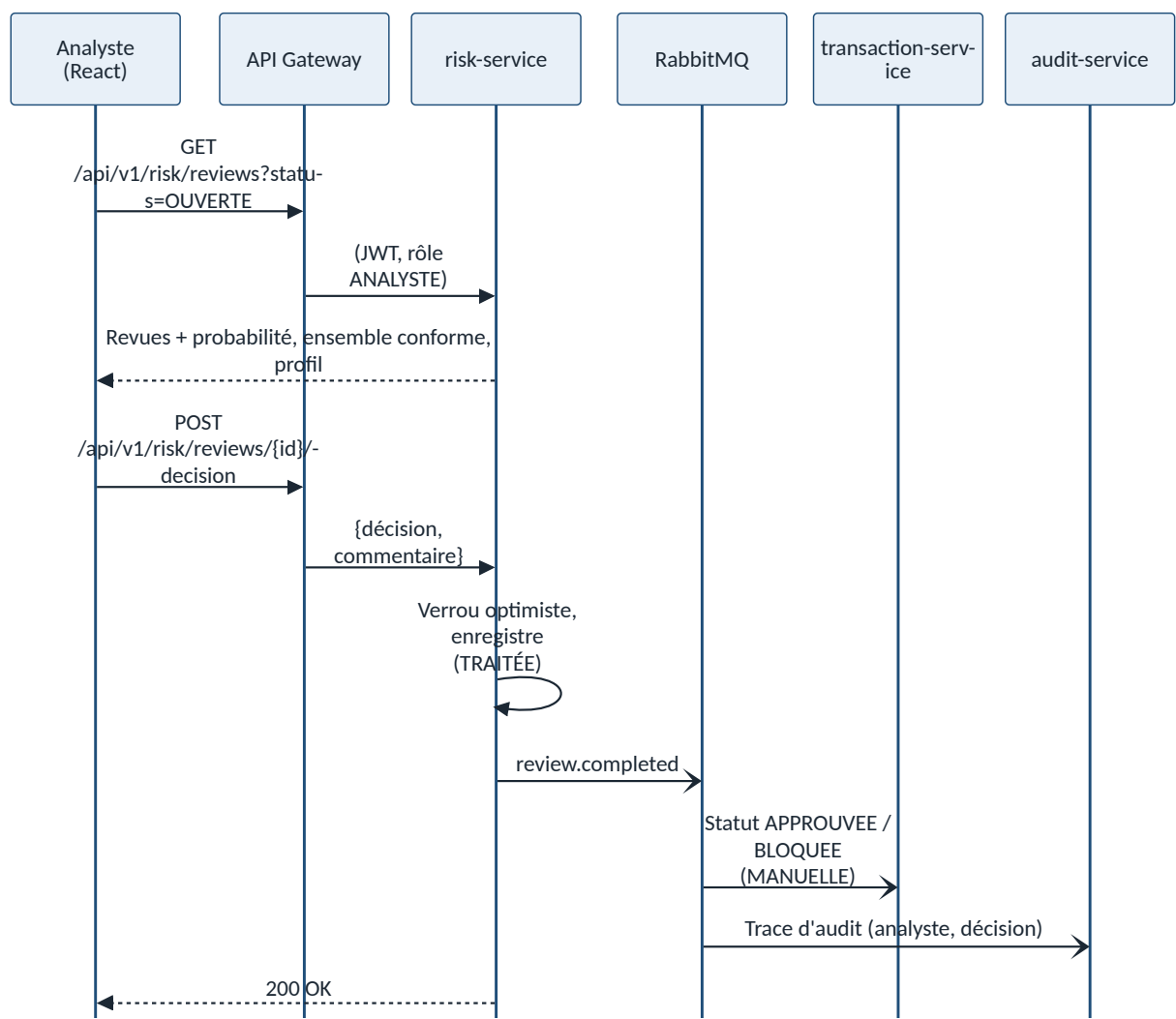
3.6 Communication synchrone et résilience

Appel	Délai max	Retry	Circuit breaker	Comportement en cas d'échec
Gateway → services	3 s	non	non	<code>503</code> au format Problem Details
transaction → risk	2 s	non	oui	Transaction laissée <code>EN_ATTENTE</code> et réponse <code>202 Accepted</code> . Un job planifié la réévalue toutes les 30 s
risk → customer	500 ms	1 (GET)	oui	Utilise le cache. Sans cache, <code>REVUE HUMAINE</code>
risk → ml-service	800 ms	1 (erreur de connexion)	oui	<code>REVUE HUMAINE</code> , raison <code>ML_INDISPONIBLE</code> (fail-safe)

Principe. Aucune panne ne conduit à approuver automatiquement une transaction. Au pire, elle attend ou elle part en revue humaine.

Réglage des circuit breakers. Le circuit s'ouvre à 50 % d'échecs sur les 20 derniers appels et reste ouvert 30 s. Son état est exposé à Prometheus et visible dans Grafana.

3.7 Flux de revue humaine



3.8 Sécurité

Matrice des droits :

Ressource	ADMIN	ANALYSTE	SIMULATEUR
Utilisateurs (CRUD)	✓	-	-
Clients (lecture)	✓	✓	-
Clients (écriture)	✓	-	-
Transactions (création)	-	-	✓
Retour d'étiquettes (feedback)	-	-	✓
Transactions (lecture)	✓	✓	-
File de revue et décision	-	✓	-
Notifications	✓	✓	-
Journal d'audit	✓	-	-
Visualisations ML, infos modèle	✓	✓	-

Mesures :

- **Défense en profondeur.** La Gateway valide le JWT, et chaque service le **revalide** en tant que resource server Spring Security.
- **Endpoints internes.** Les chemins `/internal/**` ne sont pas routés par la Gateway. Ils seront aussi protégés par des **NetworkPolicies** Kubernetes (section 6).
- **HTTPS.** TLS est terminé à l'Ingress avec un certificat local (mkcert).
- **Secrets.** Mots de passe de base de données, clé privée JWT et identifiants RabbitMQ sont stockés dans des Secrets Kubernetes, jamais dans le dépôt.
- **Données personnelles.** Le numéro de carte n'est jamais stocké en clair, et les données personnelles sont masquées dans les logs.

3.9 Données (PostgreSQL)

Schéma	Utilisateur	Service	Tables principales	Droits
auth	auth_user	auth-service	users, refresh_tokens, outbox_events	complets sur son schéma
customer	customer_user	customer-service	customers	complets sur son schéma
transaction	transaction_user	transaction-service	transactions, outbox_events, processed_events	complets sur son schéma
risk	risk_user	risk-service	assessments, reviews, feedback, outbox_events, processed_events	complets sur son schéma
notification	notification_user	notification-service	notifications, processed_events	complets sur son schéma
audit	audit_user	audit-service	audit_events	INSERT + SELECT uniquement
mlflow	mlflow_user	MLflow	tables internes de MLflow	complets sur son schéma

- **Aucun droit croisé.** Chaque utilisateur n'a **aucun droit** sur les autres schémas.
- **Migrations.** Chaque service gère son schéma avec ses propres migrations **Flyway**, versionnées avec son code.
- **Idempotence.** Les tables `processed_events` mémorisent les `eventId` déjà traités.
- **Outbox.** Les tables `outbox_events` contiennent les événements à publier (section 3.5).

3.10 Conventions transverses

Sujet	Convention
Versionnement des API	Préfixe <code>/api/v1</code> . Contrats écrits d'abord (décision D32) : les fichiers OpenAPI YAML sont rédigés en S1-S2 et versionnés dans <code>contracts/</code> , puis le code les implémente. springdoc génère le contrat depuis le code, et la CI vérifie qu'il correspond au fichier versionné (section 7.4)
Format des erreurs	Problem Details (RFC 7807) , natif dans Spring Boot
Pagination	<code>?page=&size=&sort=</code> , réponse avec métadonnées de page
Santé	<code>/actuator/health/liveness</code> et <code>/actuator/health/readiness</code> , utilisés par les probes Kubernetes
Métriques	<code>/actuator/prometheus</code> (Micrometer) : requêtes, latence, erreurs, JVM, pool de connexions, circuit breakers, RabbitMQ
Logs	Format JSON sur la sortie standard, avec <code>correlationId</code> et nom du service

Sujet	Convention
Tests	Tests unitaires avec JUnit 5 et Mockito. Tests d'intégration avec Testcontainers (PostgreSQL, RabbitMQ), exécutés en CI

4. Architecture ML : données, benchmark, validation statistique et incertitude

Responsable : P2 (Data Scientist). **Support :** P3 (intégration au pipeline et à MLflow).

Principe directeur : **le protocole expérimental est fixé avant de lancer les expériences**. Les découpages, la métrique principale, les tests et la règle de sélection du modèle sont décrits ici, puis appliqués sans modification. Cela évite d'ajuster la méthode après avoir vu les résultats.

4.1 Jeu de données : Sparkov

Jeu synthétique de transactions par carte bancaire, fourni en deux fichiers déjà découpés dans le temps :

Fichier	Contenu	Usage
fraudTrain.csv	Période la plus ancienne (~1,3 M transactions)	Entraînement, benchmark, calibration
fraudTest.csv	Période suivante (~0,56 M transactions)	Test final uniquement , jamais utilisé avant l'évaluation finale

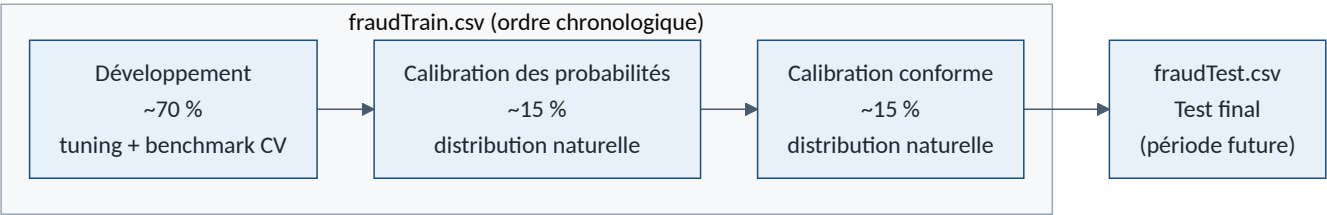
Le taux de fraude est **inférieur à 1 %**, ce qui en fait un problème fortement déséquilibré. Les chiffres exacts seront confirmés par P2 lors de l'analyse exploratoire.

Colonnes et usage :

Catégorie	Colonnes	Usage
Identifiants	trans_num , cc_num , unix_time	Exclus du modèle. trans_num sert de clé d'idempotence, cc_num est haché par customer-service
Données personnelles	first , last , street , zip	Exclues : aucune valeur prédictive légitime
Transaction	trans_date_trans_time , amt , category , merchant , merch_lat , merch_long	Features
Client	dob , gender , job , city , state , city_pop , lat , long	Features (voir 4.3 pour les attributs sensibles)
Cible	is_fraud	Étiquette

4.2 Découpage des données

Le découpage respecte **l'ordre chronologique**, pour reproduire la réalité : on apprend sur le passé et on prédit le futur.



Partie	Rôle	Règle
Développement	Réglage des hyperparamètres et benchmark par validation croisée	Seule partie utilisée pour comparer les 5 algorithmes

Partie	Rôle	Règle
Calibration des probabilités	Ajuste les probabilités du modèle final (isotonique ou sigmoïde)	Jamais vue pendant l'entraînement
Calibration conforme	Calcule les seuils de la prédiction conforme	Jamais vue pendant l'entraînement ni la calibration des probabilités
Test final	Mesure unique et définitive de la performance, de la couverture et du taux de revue	Utilisé une seule fois

Les deux calibrations utilisent des données différentes, pour que l'une ne fausse pas les garanties de l'autre.

4.3 Features et prétraitement

Tout le prétraitement est dans un **Pipeline** **scikit-learn** (décision D13). Le même objet transforme les données à l'entraînement et en production, dans le ml-service.

Feature dérivée	Calcul
<code>amount_log</code>	$\log(1 + \text{montant})$
<code>hour</code> , <code>day_of_week</code> , <code>is_weekend</code> , <code>is_night</code>	Extraits de la date et de l'heure de la transaction
<code>age</code>	Âge du client à la date de la transaction
<code>gender</code>	Encodage binaire
<code>distance_km</code>	Distance haversine entre le client et le marchand
<code>category</code>	Encodage one-hot (14 catégories)
<code>city_pop_log</code>	$\log(\text{population de la ville})$
<code>state</code>	Encodage fréquentiel ou one-hot, selon l'analyse exploratoire

Exclus :

- **merchant** et **job** : forte cardinalité et risque de sur-apprentissage. Ils pourront être réintroduits avec un encodage par la cible si l'analyse exploratoire le justifie.
- **Features d'historique** (nombre de transactions du client sur les dernières 24 h, montant moyen, etc.) : **non incluses** (décision D17). Le ml-service reste sans état et ne connaît qu'une transaction à la fois. Sparkov est déjà bien séparable avec les features propres à chaque transaction, donc le gain attendu ne justifie pas la complexité d'un stockage d'historique. C'est documenté comme une limite.

Attributs sensibles. Le **genre** et l'**âge** sont **conservés** (décision D18). Les données sont synthétiques et il s'agit de détection de fraude, pas d'octroi de crédit. En contrepartie, un **audit d'équité** est réalisé sur le test final : rappel, taux de faux positifs et taux de revue sont comparés par genre et par tranche d'âge. Tout écart important est rapporté dans le README comme une limite éthique.

Normalisation. **StandardScaler** pour les modèles sensibles à l'échelle : régression logistique, SVM et MLP.

4.4 Gestion du déséquilibre des classes

Stratégie **identique pour tous les modèles**, pour une comparaison équitable :

1. **Sous-échantillonnage** de la classe majoritaire **dans les plis d'entraînement uniquement**. Le ratio de départ est de 1 fraude pour 10 légitimes, ajustable lors du tuning. Les plis de validation, la calibration et le test restent à la **distribution naturelle**.

2. **Correction de l'a priori** (Dal Pozzolo et al., 2015). Le sous-échantillonnage gonfle artificiellement les probabilités de fraude. Elles sont corrigées par la formule $p = \beta \cdot p_s / (\beta \cdot p_s - p_s + 1)$, où β est le taux d'échantillonnage de la classe majoritaire.

Deux avantages :

- les probabilités restent interprétables ;
- les modèles coûteux (SVM, MLP) deviennent entraînaables en un temps raisonnable, puisque chaque entraînement porte sur quelques dizaines de milliers de lignes au lieu d'un million.

4.5 Modèles comparés

Modèle	Implémentation	Remarque
Régression logistique	scikit-learn	Référence simple et interprétable
Random Forest	scikit-learn	-
Gradient Boosting	XGBoost	Candidat attendu comme le plus performant
SVM	Approximation de noyau (Nystroem) + LinearSVC	Un SVC à noyau RBF exact serait trop lent sur ce volume. L'approximation garde un comportement non linéaire. Les scores sont convertis en probabilités par la calibration
MLP	scikit-learn MLPClassifier	Arrêt anticipé (early stopping) activé

Réglage des hyperparamètres.

- Recherche aléatoire avec Optuna, avec **le même budget pour chaque modèle** (par exemple 30 essais), sur une partie temporelle distincte au début de la zone de développement.
- Les configurations retenues sont **figées** avant le benchmark.
- Le réglage n'est pas refait dans chaque pli : une validation croisée imbriquée serait trop coûteuse. C'est documenté comme une limite.

4.6 Métriques

Métrique	Pourquoi	Rôle
PR-AUC (average precision)	La plus informative pour une classe rare : elle ignore les nombreux vrais négatifs	Métrique principale (tests confirmatoires)
ROC-AUC	Capacité de classement globale	Secondaire
MCC	Résumé équilibré de la matrice de confusion, robuste au déséquilibre	Secondaire
Précision, rappel, F1	Lecture métier : fausses alertes et fraudes manquées	Secondaires
Accuracy	Demandée par le sujet. Trompeuse ici : un modèle qui prédit toujours « légitime » dépasse 99 %	Rapportée avec cet avertissement
Calibration : Brier score, ECE, diagrammes de fiabilité	La décision et l'incertitude reposent sur des probabilités fiables	Critère de sélection secondaire

Métrique	Pourquoi	Rôle
Temps d'inférence	Contrainte de production (< 50 ms, section 1.4)	p50 et p95 sur 1 000 appels d'une seule ligne, pipeline complet, CPU, 1 thread, même machine pour tous les modèles

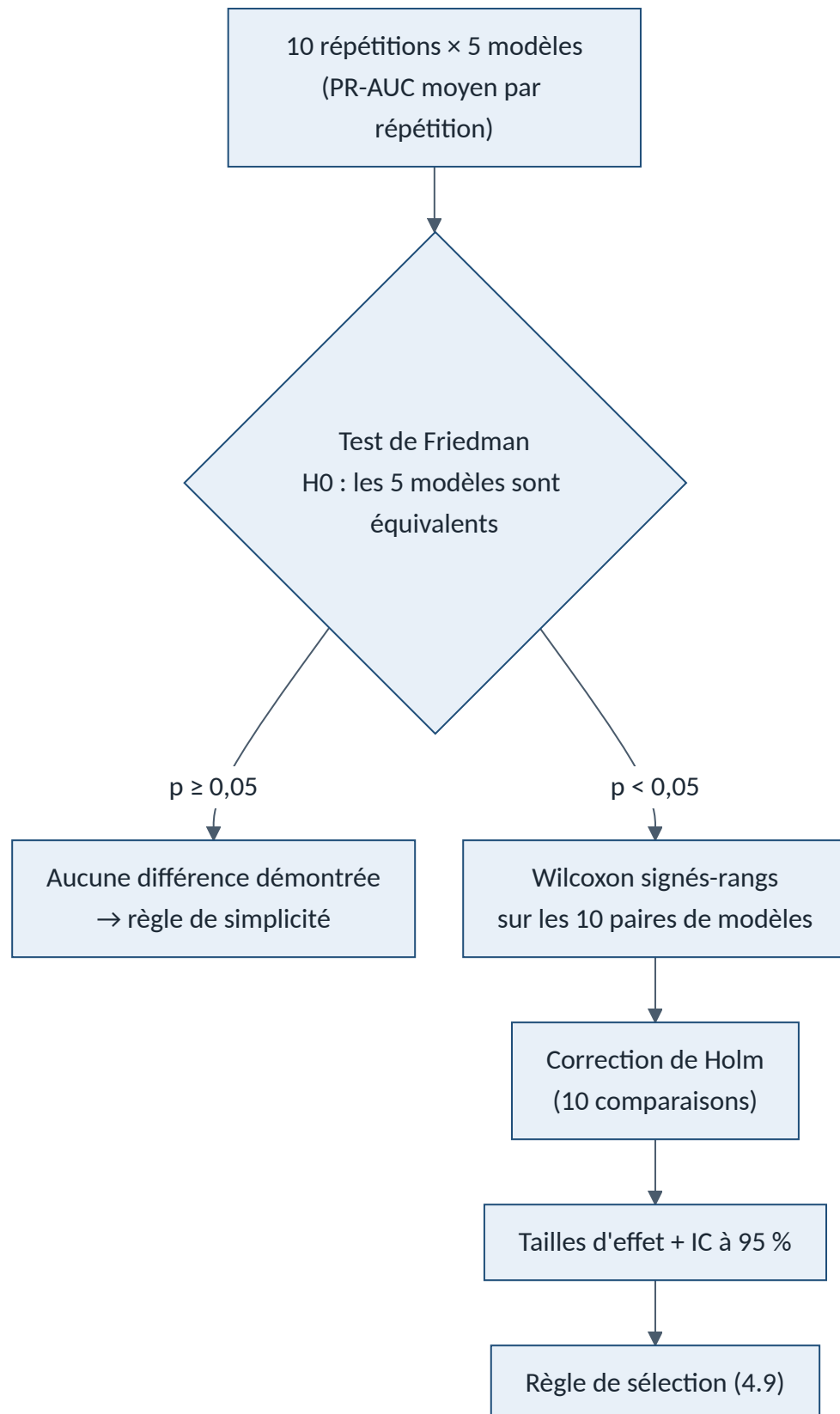
Seuil des métriques dépendantes d'un seuil (précision, rappel, F1, MCC) : il est choisi en maximisant le F1 sur une validation interne du pli d'entraînement, puis appliqué au pli de validation. Le seuil n'est donc jamais optimisé sur les données qui servent à la mesure.

Métrique métier (bonus). Coût total = montant des fraudes manquées + coût fixe par fausse alerte. Elle permet de traduire les résultats en impact financier.

4.7 Protocole de benchmark

- **Validation croisée stratifiée à 5 plis, répétée 10 fois** avec 10 seeds différentes : 50 mesures par modèle et par métrique.
- **Unité statistique : la répétition.** Les 5 plis d'une même répétition partagent des données d'entraînement, donc leurs scores ne sont pas indépendants. On utilise la **moyenne des 5 plis de chaque répétition**, soit **n = 10 blocs** par modèle. Cela évite de gonfler artificiellement la significativité.
- **Traçabilité.** Chaque pli est journalisé dans MLflow : seed, pli, hyperparamètres, métriques et durée d'entraînement.

4.8 Validation statistique



Élément	Méthode
Test global	Friedman sur les rangs des 5 modèles dans les 10 blocs. Taille d'effet : W de Kendall
Comparaisons par paires	Wilcoxon signés-rangs appariés sur les 10 blocs, pour les 10 paires

Élément	Méthode
Correction des comparaisons multiples	Holm-Bonferroni , contrôle du risque global à 5 %
Tailles d'effet par paire	Corrélation rang-bisériale (associée au Wilcoxon) et différence moyenne de PR-AUC
Intervalle de confiance	IC à 95 % par bootstrap (10 000 rééchantillonnages) sur les 10 moyennes de répétition, pour chaque modèle et chaque différence
Test de sensibilité	Test t corrigé de Nadeau et Bengio pour les paires clés. Il tient compte du chevauchement des données de la validation croisée et confirme ou nuance les conclusions
Visualisation	Diagramme de différence critique, avec les modèles classés par rang moyen
Test final	IC à 95 % par bootstrap sur fraudTest (rééchantillonnage des transactions) pour toutes les métriques du modèle retenu

Portée des conclusions.

- Les tests sur la **PR-AUC** sont **confirmatoires** : ils répondent à la question posée à l'avance.
- Les mêmes tests appliqués aux métriques secondaires sont rapportés comme **exploratoires**, pour ne pas multiplier les tests jusqu'à trouver un résultat significatif.

Limites documentées :

- Un seul jeu de données : les conclusions valent pour Sparkov et ne se généralisent pas automatiquement.
- Avec $n = 10$ blocs, la plus petite valeur p possible du Wilcoxon est d'environ 0,002. La correction de Holm reste franchissable, mais seulement pour des écarts très réguliers.
- La validation croisée est stratifiée et non temporelle. Le réalisme temporel est vérifié sur le test final.

4.9 Règle de sélection du modèle final

Cette règle est fixée **avant** les expériences :

1. Classer les modèles par PR-AUC moyenne.
2. Si le premier est **significativement meilleur** que le deuxième (p ajusté par Holm $< 0,05$, avec une taille d'effet non négligeable), **il est retenu**.
3. Sinon, parmi les modèles statistiquement équivalents au premier, choisir celui qui a la **meilleure calibration** (Brier score), puis, en cas d'égalité, **le temps d'inférence le plus court**.

Cette règle garantit que le modèle retenu est défendable, même si les différences sont faibles.

4.10 Calibration et quantification de l'incertitude

Étape 1 : calibration des probabilités. Le modèle final est entraîné sur la zone de développement, puis calibré sur la zone de calibration des probabilités. On compare **isotonique** et **sigmoïde (Platt)**, et on retient la méthode au meilleur Brier score. Les diagrammes de fiabilité avant et après calibration sont journalisés dans MLflow.

Étape 2 : prédiction conforme (MAPIE). Elle est calculée sur la zone de calibration conforme.

- La prédiction conforme renvoie un **ensemble de prédiction** qui contient la vraie classe avec une probabilité garantie d'au moins $1 - \alpha$, à condition que les données futures ressemblent aux données de calibration.
- **Conforme conditionnel à la classe (Mondrian).** Avec 99 % de transactions légitimes, une garantie globale pourrait être respectée tout en couvrant mal la classe fraude. On calibre donc **un seuil par classe**, pour garantir la couverture **aussi sur les fraudes**. Si la version de MAPIE utilisée ne le propose pas directement, le calcul se fait en quelques lignes (quantile des scores de non-conformité par classe).

Étape 3 : choix de α . On évalue $\alpha \in \{0,01 ; 0,05 ; 0,10\}$. Pour chaque valeur, on mesure :

Mesure	Signification
Couverture par classe	Vérifie la garantie $1 - \alpha$, pour les légitimes et pour les fraudes
Taux de revue	Part des transactions envoyées à un analyste (ensemble $\{0,1\}$ ou vide)
Taux d'automatisation	Part des décisions prises sans humain
Taux d'erreur des décisions automatiques	Erreurs parmi les transactions approuvées ou bloquées automatiquement

Le α retenu est **le plus petit qui maintient le taux de revue sous une capacité d'analyse réaliste**, par exemple $\leq 2\%$ des transactions. Le compromis est présenté sous forme de courbe dans le README.

Preuve attendue. Sur le test final, montrer que les erreurs du modèle se **concentrent dans les transactions envoyées en revue**, et que les décisions automatiques sont nettement plus fiables. C'est la démonstration que la revue humaine sert réellement à quelque chose.

Ce que le modèle renvoie (contrat détaillé en section 5) :

Champ	Contenu
<code>fraud_probability</code>	Probabilité calibrée de fraude
<code>prediction_set</code>	<code>[0]</code> , <code>[1]</code> , <code>[0, 1]</code> ou <code>[]</code>
<code>uncertainty</code>	Score continu entre 0 et 1 (entropie binaire normalisée de la probabilité calibrée), affiché dans le tableau de bord
<code>uncertainty_level</code>	FAIBLE (ensemble à un élément) ou ELEVEE (sinon), qui pilote la décision
<code>alpha</code>	Niveau de risque utilisé
<code>model_version</code>	Nom et version dans le Model Registry

Suivi en production. La garantie conforme suppose que les données ne dérivent pas. La couverture réelle est donc surveillée à partir des étiquettes fournies par les analystes (section 3.4). Une baisse de couverture déclenche une alerte (section 8).

4.11 Visualisation des données (PCA, t-SNE, UMAP)

Élément	Choix
Échantillon	~20 000 transactions de la zone de test : toutes les fraudes et un échantillon aléatoire de légitimes, avec des poids pour ne pas fausser la lecture
Entrée	Features après prétraitement (sortie du Pipeline)
Méthodes	PCA (linéaire, variance expliquée), t-SNE (structure locale), UMAP (structure locale et globale, plus rapide)
Vues produites	Coloration par classe, par clusters (k-means ou HDBSCAN), par score d'anomalie (Isolation Forest), par type d'erreur du modèle (vrai positif, faux positif, faux négatif), par niveau d'incertitude
Format de sortie	JSON (coordonnées et attributs de chaque point) + PNG, servis par le ml-service et affichés dans React
Régénération	Étape du pipeline ML : les vues sont recalculées à chaque nouvelle version du modèle

Avertissement affiché dans le tableau de bord. Avec t-SNE et UMAP, les distances entre les groupes et la taille des groupes n'ont pas de signification quantitative. Seule la proximité locale est interprétable.

4.12 Reproductibilité

- **Seeds fixées et journalisées.** 10 seeds pour le benchmark et une seed pour le modèle final.
- **Version des données.** L'empreinte SHA-256 de chaque fichier CSV est enregistrée dans chaque run MLflow.
- **Environnement.** Dépendances Python figées (`requirements.lock`) et même image Docker pour l'entraînement en CI.
- **Code des tests statistiques.** Il est écrit sous forme de **module Python réutilisable** (`ml/stats/validate.py` , exécutable en ligne de commande) et non uniquement dans un notebook. P3 peut ainsi l'intégrer au pipeline automatisé (section 5).
- **Rapport statistique.** Il est généré automatiquement (Markdown + figures) et stocké comme artefact MLflow du run de validation.

4.13 Livrables de la section

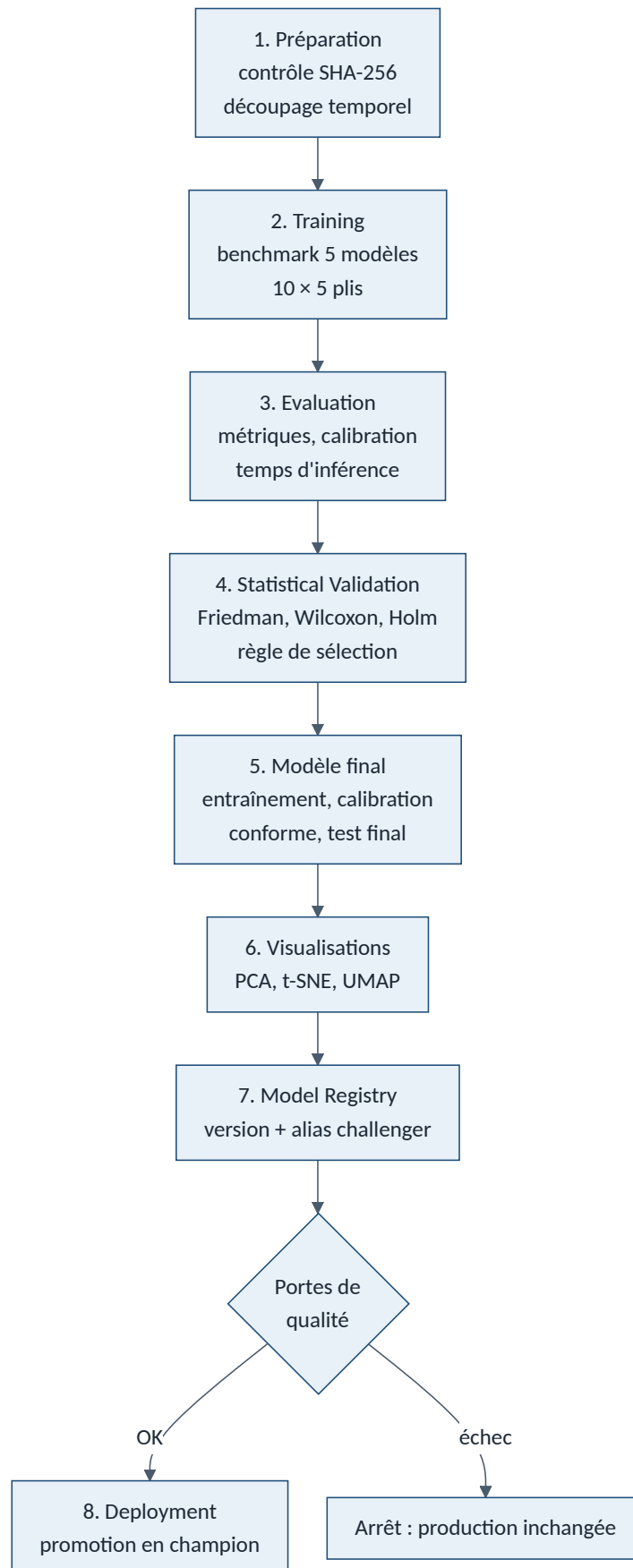
Livrable	Responsable	Semaine
Analyse exploratoire + pipeline de prétraitement	P2	S3
Benchmark des 5 modèles (10 × 5 plis)	P2	S6
Module de validation statistique + rapport	P2	S7-S8
Modèle final calibré + conforme + choix de α	P2	S8-S9
Audit d'équité (genre, tranches d'âge)	P2	S9
Script de visualisation (étape du pipeline)	P3	S8
Intégration dans le pipeline MLflow	P3	S8-S10

5. Architecture MLOps et serving du modèle

Responsable : P3 (Ingénieur MLOps). **Support** : P2 (code ML et statistique), P1 (intégration avec risk-service), P4 (CI et déploiement).

5.1 Cycle de vie du modèle

Le sujet impose la chaîne **Training → Evaluation → Statistical Validation → Model Registry → Deployment**. Chaque étape est automatisée et journalisée dans MLflow. Le passage d'une étape à la suivante est contrôlé par des **portes de qualité** : si une porte échoue, le pipeline s'arrête et le modèle en production reste inchangé.



Deux modes d'exécution :

Mode	Étapes	Usage	Durée estimée
full	1 → 8	Première exécution, ou changement de features ou d'algorithmes	30 à 90 min

Mode	Étapes	Usage	Durée estimée
retrain	1, 5 → 8	Réentraînement de l'algorithme déjà sélectionné (nouvelles données, nouveau α)	5 à 15 min

5.2 Serveur MLflow

Élément	Choix
Déploiement	Deployment Kubernetes (1 replica), namespace <code>ml</code>
Backend store (runs, métriques, registry)	PostgreSQL, schéma <code>mlflow</code> (décision D5)
Artefacts (modèles, figures, rapports)	Volume persistant (PVC), servi par MLflow lui-même (<code>--serve-artifacts</code>). Pas de MinIO, pour économiser la RAM
Accès	<code>https://mlflow.fintech.local</code> via l'Ingress, réservé à l'équipe
Registry	Modèle enregistré <code>fraud-detector</code> , avec des <code>alias</code> <code>champion</code> (production) et <code>challenger</code> (candidat). Les « stages » sont obsolètes depuis MLflow 2.9

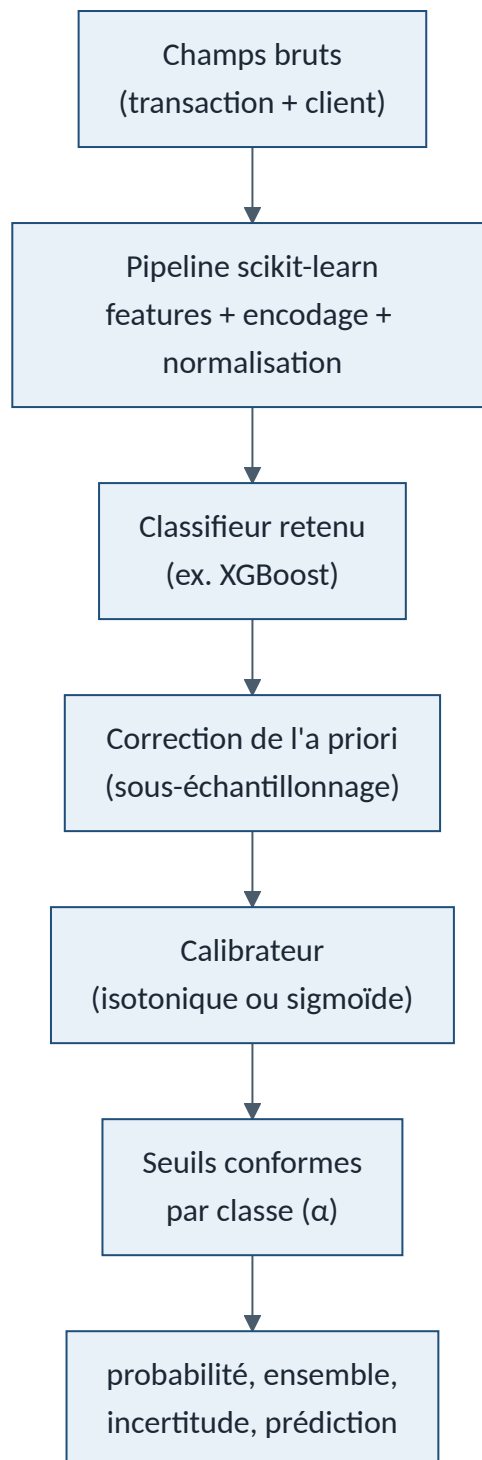
Organisation des runs. Chaque exécution du pipeline crée un **run parent**, et chaque étape un **run enfant**. Chaque pli du benchmark est aussi un run enfant de l'étape Training. Tout est ainsi consultable dans l'interface, de l'exécution globale jusqu'au pli individuel.

Ce qui est journalisé :

Type	Contenu
Paramètres	Hyperparamètres, seed, pli, ratio de sous-échantillonnage, α , méthode de calibration
Métriques	Toutes les métriques de la section 4.6, par pli, par répétition et agrégées
Tags	Commit Git, empreinte SHA-256 des données, mode du pipeline, image Docker utilisée
Artefacts	Modèle packagé, diagrammes de fiabilité, courbes PR et ROC, diagramme de différence critique, rapport statistique (Markdown), courbe α / taux de revue, audit d'équité, visualisations JSON et PNG

5.3 Packaging du modèle

Le modèle est enregistré comme un **modèle MLflow `pyfunc` personnalisé** qui regroupe tout ce qui est nécessaire à une prédiction :



- **Un seul artefact, un seul appel.** Le ml-service charge un seul objet et appelle `predict`. Il ne connaît aucun détail du modèle.
- **Signature MLflow.** Le schéma des champs d'entrée est enregistré avec le modèle, ce qui permet de détecter une incompatibilité dès le chargement.
- **Visualisations liées au modèle.** Les fichiers de visualisation et le résumé des métriques sont attachés à la **même version**. Le tableau de bord affiche donc toujours des vues cohérentes avec le modèle en production.
- **Traçabilité complète.** Chaque version porte les tags `git_commit`, `data_sha256`, `pipeline_run_id` et `alpha`. On peut remonter d'une décision de l'audit jusqu'au code et aux données d'entraînement.

5.4 Portes de qualité avant promotion

Porte	Condition	Pourquoi
Non-régression	PR-AUC du candidat sur le test $\geq$ PR-AUC du champion - 0,01	Ne jamais déployer un modèle moins bon
Couverture conforme	Couverture par classe $\geq (1 - \alpha) - 0,02$ sur le test	La garantie d'incertitude doit tenir
Charge de revue	Taux de revue $\leq 2\%$	Capacité réaliste des analystes
Latence	Inférence p95 < 50 ms (pipeline complet, 1 thread)	Exigence non fonctionnelle (section 1.4)
Test de fumée	Le modèle se charge depuis le registry et prédit correctement 100 transactions de référence	Détecte un artefact corrompu ou une signature incompatible

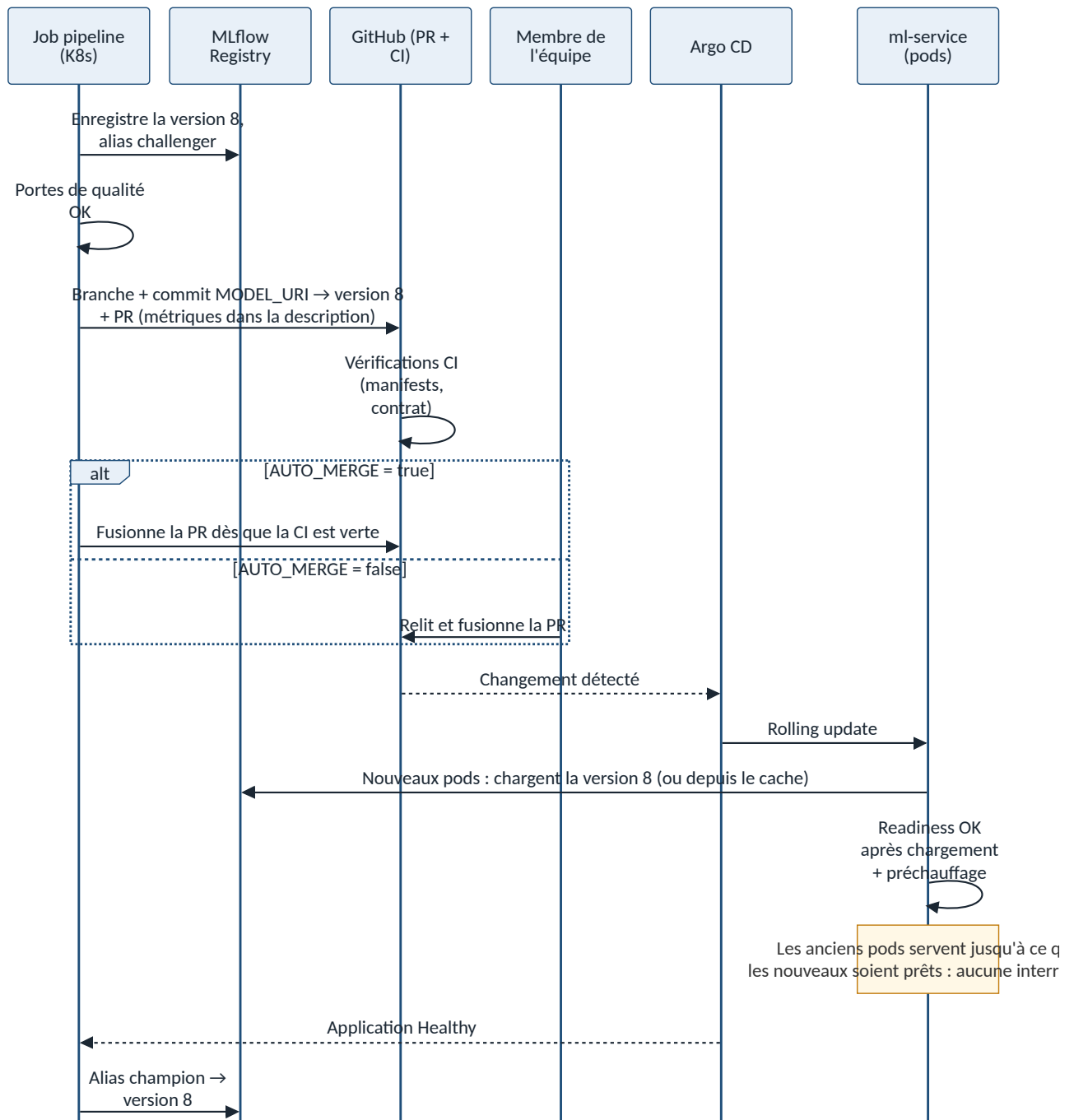
À la première exécution, sans champion, la porte de non-régression est remplacée par un minimum absolu : PR-AUC $\geq$ celle de la régression logistique.

5.5 Déploiement d'une nouvelle version du modèle (décision D20)

Principe : la version du modèle en production est écrite dans Git. La ConfigMap du ml-service contient `MODEL_URI=models:/fraud-detector/<version>`, une version **exacte** et non un alias. Argo CD déploie donc le modèle exactement comme il déploie le code.

Promotion par PR, fusionnée automatiquement si tout est vert.

- Le pipeline ouvre une PR qui change la version.
- Les vérifications de la CI s'exécutent sur cette PR.
- Selon le paramètre `AUTO_MERGE`, le pipeline fusionne lui-même la PR, ou un membre de l'équipe la valide.



AUTO_MERGE	Usage	Argument
true	Développement et démo	Déploiement entièrement automatique, conforme à l'exigence « automated deployment » du sujet
false	Mode « production » documenté	Validation à quatre yeux, une bonne pratique en finance

Avantages :

- **Un seul mécanisme de déploiement** (Argo CD) pour le code, la configuration et le modèle, et une seule source de vérité : Git.
- **Rollback du modèle** = `git revert` de la PR. Argo CD redéploie l'ancienne version par rolling update.
- **Historique complet**. Chaque déploiement de modèle est un commit daté, avec ses métriques et, en mode manuel, son approbateur.

- **Sécurité du rolling update.** Si le nouveau modèle ne se charge pas, la readiness probe échoue et le rolling update se bloque. Les anciens pods continuent de servir, Argo CD passe l'application en `Degraded`, et le Job **ne déplace pas** l'alias `champion`.

Automatisation. Le Job utilise l'API GitHub avec un jeton stocké dans un Secret Kubernetes, limité à ce dépôt et aux droits de PR.

5.6 Où et quand le pipeline s'exécute

GitHub Actions tourne dans le cloud et ne peut pas joindre le MLflow du cluster local. L'exécution est donc répartie ainsi :

Contexte	Où	Ce qui est exécuté
CI (chaque PR)	GitHub Actions	Lint, tests unitaires du prétraitement et du module statistique, puis pipeline de fumée sur un échantillon de 1 % avec un MLflow local au runner. Cela prouve que le pipeline fonctionne de bout en bout
Exécution officielle	Job Kubernetes dans le cluster, image <code>ml-pipeline:<sha></code> construite par la CI	Pipeline complet, journalisé dans le MLflow du cluster
Exploration	Machine de P2	Notebooks et essais. Les résultats officiels proviennent toujours du Job, ce qui prouve la reproductibilité

- **Déclenchement du Job.** Un CronJob **suspendu** sert de modèle. `make pipeline MODE=full` lance `kubectl create job --from=cronjob/ml-pipeline`.
- **Ressources.** Requête de 2 CPU et 4 Gi. Le Job n'est pas lancé pendant les tests de charge.

5.7 Service FastAPI (ml-service)

Endpoints

Endpoint	Accès	Rôle
<code>POST /predict</code>	Interne (risk-service), non routé par la Gateway	Prédiction d'une transaction
<code>GET /health/live</code>	Kubernetes	Le processus répond
<code>GET /health/ready</code>	Kubernetes	Le modèle est chargé et le test de préchauffage a réussi
<code>GET /metrics</code>	Prometheus	Métriques techniques et ML
<code>GET /api/v1/ml/model</code>	Gateway (ADMIN, ANALYSTE)	Nom, version, date d'entraînement, α , algorithme, commit Git
<code>GET /api/v1/ml/performance</code>	Gateway	Métriques hors ligne : benchmark, résultats statistiques, test final avec IC, audit d'équité
<code>GET /api/v1/ml/viz/{method}?view=</code>	Gateway	Visualisations. <code>method</code> : <code>pca</code> , <code>tsne</code> ou <code>umap</code> . <code>view</code> : <code>class</code> , <code>cluster</code> , <code>outlier</code> , <code>error</code> ou <code>uncertainty</code>

Contrat de `/predict`

Requête :

```
{
  "transaction_id": "7f3c9a1e-2b4d-4c8e-9f1a-0d5e6b7c8a9f",
  "trans_datetime": "2020-07-14T02:13:45",
  "amount": 842.17,
  "category": "shopping_net",
  "merch_lat": 40.71,
  "merch_long": -74.01,
  "customer": {
    "dob": "1962-05-18",
    "gender": "F",
    "city_pop": 1577385,
    "lat": 40.65,
    "long": -73.95,
    "state": "NY"
  }
}
```

Réponse :

```
{
  "transaction_id": "7f3c9a1e-2b4d-4c8e-9f1a-0d5e6b7c8a9f",
  "prediction": 1,
  "fraud_probability": 0.87,
  "prediction_set": [1],
  "uncertainty": 0.56,
  "uncertainty_level": "FAIBLE",
  "alpha": 0.05,
  "model": { "name": "fraud-detector", "version": "8" },
  "inference_ms": 4.2
}
```

Validation des entrées. Le modèle Pydantic vérifie :

- montant strictement positif ;
- catégorie parmi les 14 connues ;
- latitude et longitude dans leurs bornes ;
- dates cohérentes.

Une entrée invalide renvoie `422` , et un compteur Prometheus est incrémenté.

Séparation des responsabilités. Le ml-service **ne décide pas**. Il fournit la prédiction et son incertitude, et risk-service applique la politique de décision (section 2.5). La politique peut donc évoluer sans redéployer le modèle, et inversement.

Mode mock (livré en S6). La même image, lancée avec `MODEL_MODE=mock` , renvoie des réponses déterministes qui respectent exactement le contrat. Par exemple, montant > 500 donne `[1]` , montant entre 200 et 500 donne `[0,1]` . P1 développe et teste risk-service sans attendre le vrai modèle, et peut provoquer volontairement les trois décisions.

Choix d'implémentation

- **Un processus Uvicorn par pod.** La mise à l'échelle se fait par le nombre de pods (HPA), et non par des workers dans un pod. Kubernetes mesure ainsi correctement le CPU de chaque instance.
- **Endpoint `/predict` synchrone** (`def` et non `async def`). La prédiction sollicite le CPU : FastAPI l'exécute dans son pool de threads, ce qui ne bloque pas la boucle d'événements.
- **XGBoost limité à `n_jobs=1`** , pour respecter la limite CPU du pod et éviter la sur-souscription.
- **Chargement au démarrage.** Le modèle est chargé dans le `lifespan` de FastAPI, puis une prédiction de préchauffage est exécutée. La readiness ne passe à OK qu'après ces deux étapes.

- **Cache local du modèle.** Le modèle téléchargé est stocké sur le disque du nœud (volume `hostPath`, un cache par nœud), indexé par version. Un volume persistant classique ne peut pas être partagé entre des pods placés sur des nœuds différents (section 6.10). Un pod qui redémarre recharge depuis le cache sans dépendre de MLflow. Une panne de MLflow **n'empêche donc pas** les pods de redémarrer, tant que la version est en cache.

5.8 Métriques exposées par le ml-service

Métrique Prometheus	Type	Usage
<code>http_requests_total</code> , <code>http_request_duration_seconds</code>	Compteur, histogramme	Débit, latence, erreurs HTTP
<code>ml_inference_duration_seconds{model_version}</code>	Histogramme	Temps d'inférence p50 et p95
<code>ml_predictions_total{model_version, prediction_set}</code>	Compteur	Répartition <code>0</code> , <code>1</code> , <code>01</code> , <code>vide</code> , soit les taux d'automatisation et de revue
<code>ml_fraud_probability</code>	Histogramme	Distribution des scores : un changement signale une dérive
<code>ml_uncertainty</code>	Histogramme	Distribution de l'incertitude
<code>ml_input_validation_errors_total</code>	Compteur	Entrées invalides
<code>ml_model_info{name, version}</code>	Jauge (= 1)	Version en production, visible dans Grafana
<code>ml_model_loaded_timestamp_seconds</code>	Jauge	Date du dernier chargement

5.9 Suivi de la performance en production (décision D19)

Problème. Pour mesurer la performance réelle du modèle, il faut connaître la vérité sur chaque transaction. Or, en production, on ne la connaît que pour les transactions **revues par un analyste**, qui sont justement les cas incertains. On n'a aucune vérité sur les décisions automatiques.

Solution proposée : les rejets de paiement simulés. Dans la réalité, une fraude non détectée finit par être signalée par le client, plusieurs jours plus tard : c'est un *chargeback*. Le simulateur connaît la vraie étiquette `is_fraud` de chaque transaction Sparkov. Il l'envoie donc **avec un délai** (par exemple 2 minutes en accéléré), via `POST /api/v1/risk/feedback` (rôle SIMULATEUR).

risk-service rapproche chaque étiquette de la décision prise et expose des compteurs :

Métrique	Ce qu'elle permet de calculer dans Grafana
<code>risk_feedback_total{decision, truth}</code>	Matrice de confusion en production , puis précision, rappel et taux de faux positifs par fenêtre de temps
<code>risk_conformal_covered_total{truth}</code> et <code>risk_feedback_total{truth}</code>	Couverture conforme réelle par classe. Une chute signale une dérive et déclenche une alerte
<code>risk_review_agreement_total{analyst_decision, truth}</code>	Qualité des décisions humaines

Ce mécanisme alimente directement les tableaux de bord « Performance du modèle » et « Incertitude » demandés par le sujet. Il ne demande que quelques lignes dans le simulateur et un endpoint dans risk-service.

5.10 Versionnement et rollback

Élément	Versionné dans	Identifiant	Rollback
Code	Git	Commit SHA	<code>git revert</code> puis CI et Argo CD

Élément	Versonné dans	Identifiant	Rollback
Image Docker	GHCR	Tag = commit SHA	Revert du manifest (Argo CD)
Configuration	Git (ConfigMaps)	Commit SHA	<code>git revert</code>
Modèle	MLflow Registry, et version épinglée dans Git	<code>fraud-detector/<n></code>	<code>git revert</code> de la PR de promotion
Données	Empreinte dans MLflow	SHA-256	Refaire l'exécution avec le fichier correspondant
Expérience	MLflow	Identifiant du run	Consultation, comparaison

Scénario de démonstration (rollback du modèle).

1. On déploie volontairement une version dégradée, par exemple entraînée sur des données tronquées, en contournant les portes pour la démo.
2. Grafana montre la chute de la couverture et la hausse du taux de revue.
3. On fait `git revert` de la PR.
4. Argo CD revient à la version précédente et les métriques se rétablissent.

5.11 Intégration avec risk-service

Point	Mise en œuvre
Contrat	Le schéma OpenAPI généré par FastAPI (<code>/openapi.json</code>) est versionné dans le dépôt . La CI échoue si le code et le contrat divergent
Développement de P1	Mode mock du ml-service (S6) dans Docker Compose, et stubs WireMock générés depuis les exemples du contrat pour les tests Java
Résilience	Délai max de 800 ms, 1 retry, circuit breaker et repli en REVUE HUMAINE (section 3.6)
Traçabilité	<code>model.version</code> est enregistré dans <code>assessments</code> , puis propagé dans l'événement <code>transaction.decided.*</code> et dans l'audit
Corrélation	L'en-tête <code>X-Correlation-Id</code> est lu par FastAPI et ajouté à ses logs JSON

5.12 Livrables et calendrier

Livrable	Responsable	Semaine
Serveur MLflow (Docker Compose, puis Kubernetes)	P3	S1-S3
Squelette du pipeline + pipeline de fumée en CI	P3	S4-S5
ml-service en mode mock + contrat OpenAPI	P3	S6
Premier vrai modèle en production (meilleur modèle du benchmark, avant la validation statistique)	P3	S8
Étape de visualisation + endpoints <code>/api/v1/ml/viz</code>	P3	S8
Pipeline complet avec portes de qualité, promotion par PR	P3	S8-S10
ml-service final (modèle du registry, cache, métriques)	P3	S10
Endpoint de feedback et métriques de performance en production	P1 (risk-service), P3 (simulateur)	S10

6. Architecture de déploiement : Docker et Kubernetes

Responsable : P4 (DevOps). **Support** : P1 et P3 (images, probes, métriques), P5 (supervision).

6.1 Images Docker

Image	Base	Construction	Taille visée
Services Spring Boot (x7)	<code>eclipse-temurin:21-jre-alpine</code>	Multi-étapes : build Maven, puis couches Spring Boot extraites (dépendances, puis code) pour un cache Docker efficace	~200 Mo
ml-service	<code>python:3.12-slim</code>	Multi-étapes : dépendances installées dans un venv, puis copie dans l'image finale	~500 Mo
ml-pipeline	<code>python:3.12-slim</code>	Même base que ml-service, plus les dépendances d'entraînement	~1 Go
frontend	<code>nginx:alpine</code> (serveur web statique)	Multi-étapes : build Vite, puis fichiers statiques servis par nginx	~30 Mo
simulator	<code>python:3.12-slim</code>	Script de jeu de Sparkov	~150 Mo

Règles communes :

- **Exécution non-root**. Utilisateur dédié et système de fichiers en lecture seule lorsque c'est possible.
- **Tag = commit SHA**. Pas de `latest` dans les manifests : chaque déploiement est traçable et reproductible.
- **Analyse Trivy** de chaque image en CI : une vulnérabilité critique bloque la publication (section 7).
- **Docker Compose** reste disponible pour le développement local de chaque membre, avec les mêmes images.

6.2 Cluster local

kind avec 3 nœuds (décision D23) : 1 control-plane et 2 workers.

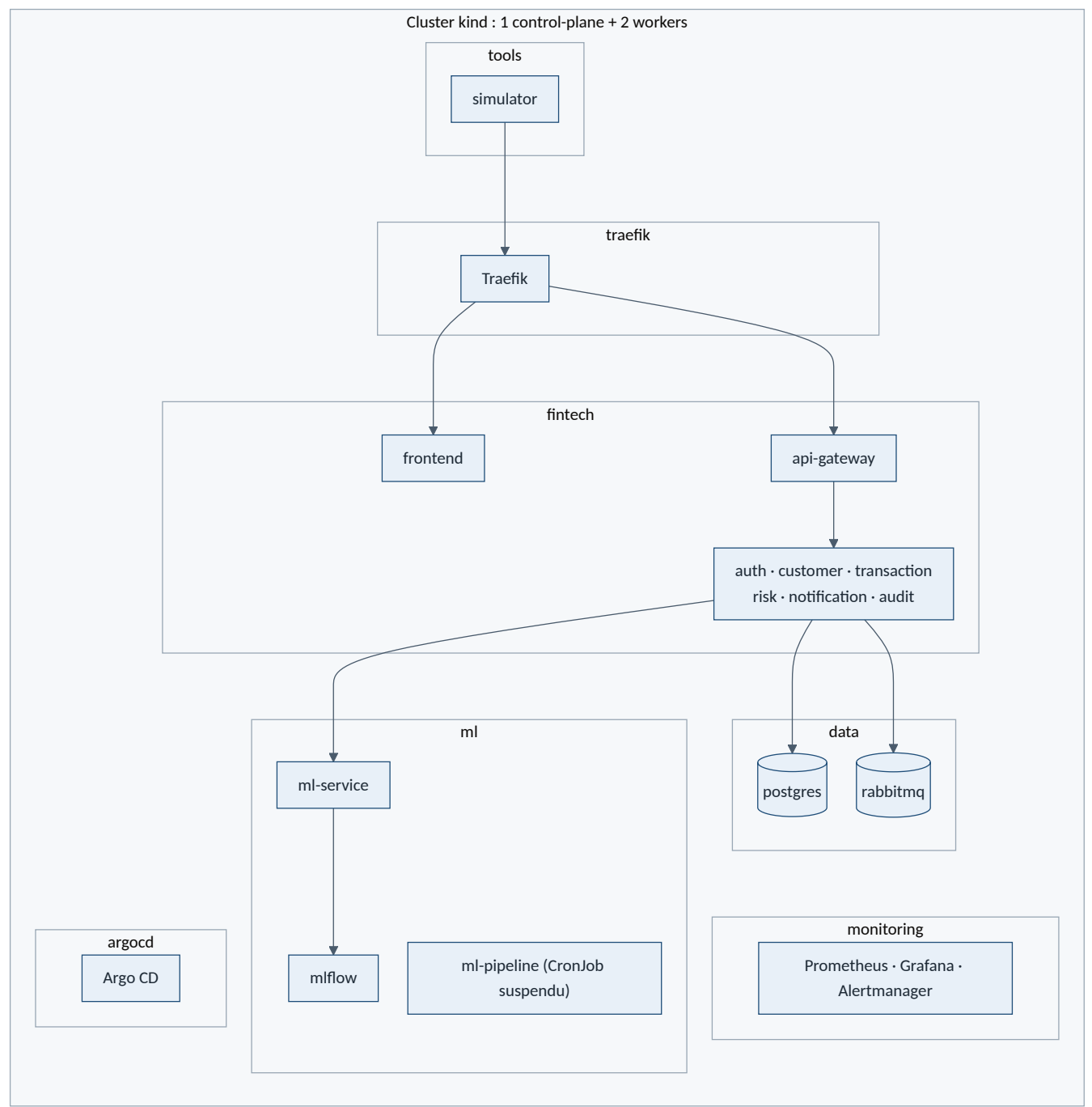
- Deux workers permettent de **répartir les replicas** sur des nœuds différents et de **simuler la panne d'un nœud** complet (`docker stop` du nœud), en plus de la suppression de pods.
- Chaque nœud kind est un conteneur Docker, donc le surcoût est faible (quelques centaines de Mo).
- **Attention** : dans kind, chaque nœud annonce la **totalité** de la RAM de la machine hôte. Le scheduler ne peut donc pas empêcher une surconsommation : la vraie limite reste les 16 Go physiques. Les requests et limits (6.5) sont dimensionnées en conséquence.

Composant du cluster	Choix	Rôle
CNI	kindnet (par défaut)	Réseau des pods. P4 vérifie en S2 que les NetworkPolicies sont bien appliquées (un pod non autorisé ne doit pas joindre risk-service). Sinon, passage à Calico
metrics-server	Option <code>--kubelet-insecure-tls</code> (nécessaire dans kind)	Fournit le CPU et la mémoire des pods à l'HPA
Contrôleur Ingress	Traefik (voir ci-dessous)	Entrée HTTP(S) du cluster
Certificats	mkcert : certificat local de confiance pour <code>*.fintech.local</code>	HTTPS sans avertissement dans le navigateur
Argo CD	Namespace <code>argocd</code>	Déploiement GitOps

Composant du cluster	Choix	Rôle
Stockage	<code>local-path</code> (fourni par kind)	Volumes persistants

Pourquoi pas Ingress NGINX. Le projet Ingress NGINX de Kubernetes a été **retiré en mars 2026** : il ne reçoit plus aucune mise à jour, y compris de sécurité. Les responsables de Kubernetes recommandent de migrer vers Gateway API ou vers un autre contrôleur Ingress. Pour une plateforme FinTech présentée en 2027, utiliser un composant abandonné serait une faiblesse. **Traefik** (décision D21) est maintenu, léger, et prend en charge la ressource `Ingress` standard demandée par le sujet.

6.3 Namespaces



Séparation par responsabilité. Chaque namespace peut porter ses propres NetworkPolicies et ses propres quotas, et être lu séparément dans Grafana.

6.4 Charges de travail

Composant	Type Kubernetes	Replicas (min-max)	HPA	Stockage
frontend	Deployment	2	non	-
api-gateway	Deployment	2-3	oui	-
auth-service	Deployment	1-2	oui	-
customer-service	Deployment	1-2	oui	-
transaction-service	Deployment	2-4	oui	-
risk-service	Deployment	2-4	oui	-
notification-service	Deployment	1-2	oui	-
audit-service	Deployment	1	non	-
ml-service	Deployment	2-5	oui	cache hostPath
mlflow	Deployment	1	non	PVC 5 Gi
postgres	StatefulSet	1	non	PVC 5 Gi
rabbitmq	StatefulSet	1	non	PVC 1 Gi
ml-pipeline	CronJob (suspendu)	-	-	-
simulator	Deployment (débit configurable, réductible à 0)	1	non	-

Principes :

- **Chemin critique toujours en double.** Gateway, transaction, risk et ml-service ont au moins 2 replicas : la suppression d'un pod n'interrompt jamais le traitement des transactions.
- **Services non critiques à 1 replica.**
 - Une panne d'auth n'empêche pas le traitement des transactions : les JWT restent validés grâce à la clé publique en cache.
 - Une panne d'audit ou de notification n'est pas bloquante non plus : RabbitMQ garde les messages en attente.
- **Maximums HPA volontairement bas,** calculés pour rester dans les 16 Go (6.5).

Limite documentée : PostgreSQL et RabbitMQ ne sont pas redondants. Un cluster PostgreSQL (CloudNativePG, Patroni) ou RabbitMQ serait trop lourd pour la machine. Les risques sont atténués ainsi :

- **Données persistantes** sur PVC : un pod supprimé redémarre avec ses données en quelques secondes.
- **L'outbox (section 3.5) protège contre une panne de RabbitMQ.** Les événements s'accumulent dans PostgreSQL et sont publiés au retour du broker. Les expériences du chapitre 9 mesurent ce temps de récupération.
- **Placement fixe.** Les composants avec état sont placés sur **worker-1** (nodeAffinity), car un volume **local-path** est lié à son nœud. La simulation de panne de nœud cible **worker-2**.

6.5 Ressources (requests et limits)

Composant	CPU request	CPU limit	Mémoire request	Mémoire limit
Service Spring Boot (chacun)	200m	1000m	384 Mi	512 Mi
ml-service	250m	1000m	384 Mi	768 Mi

Composant	CPU request	CPU limit	Mémoire request	Mémoire limit
frontend	50m	200m	64 Mi	128 Mi
postgres	250m	1000m	512 Mi	1 Gi
rabbitmq	200m	500m	384 Mi	512 Mi
mlflow	100m	500m	384 Mi	768 Mi
simulator	50m	200m	64 Mi	128 Mi

JVM. On utilise `-XX:MaxRAMPercentage=70` plutôt qu'un `-Xmx` fixe. La taille du tas suit ainsi automatiquement la limite mémoire du conteneur (~360 Mo sur 512 Mi), en laissant de la place au métaspace et aux threads.

Budget mémoire (requests) :

Situation	Pods applicatifs	Mémoire demandée (applicatif + données + ML)	Avec plateforme (K8s, Traefik, Argo CD, monitoring, Loki)
Au repos (minimums)	15	~6 Gi	~9,5 Gi
Au maximum de l'HPA	26	~10 Gi	~13,5 Gi

Au maximum de l'HPA, il ne reste qu'environ 2,5 Go pour le système d'exploitation. C'est serré mais faisable : k6 s'exécute sur une autre machine (décision D28), et l'IDE et le navigateur sont fermés pendant les tests de charge. Si la mémoire manque, les maximums de l'HPA seront réduits en priorité sur le ml-service (2-4 au lieu de 2-5). Les requests étant des réservations, la consommation réelle est souvent plus basse ; elle sera mesurée en S8.

6.6 Autoscaling (HPA)

Paramètre	Valeur	Justification
API	<code>autoscaling/v2</code>	Version stable avec contrôle du comportement de mise à l'échelle
Métrique	Utilisation CPU, cible 70 %	L'HPA agit avant l'alerte CPU > 80 % du sujet
Montée	Aucune stabilisation, jusqu'à +100 % de pods toutes les 30 s	Réagir vite à un pic
Descente	Stabilisation de 300 s, -1 pod par minute	Éviter les oscillations (flapping)

Pourquoi pas la mémoire. Le tas d'une JVM grandit mais ne rend presque jamais la mémoire : un HPA sur la mémoire monterait sans jamais redescendre.

Démarrage des pods Java. Les pics CPU au démarrage des pods Java ne déclenchent pas de mise à l'échelle parasite : l'HPA ignore le CPU des pods qui ne sont pas encore prêts.

Bonus possible. Un HPA sur le débit de requêtes via Prometheus Adapter. Il n'est pas prévu dans la version de base.

6.7 Sondes de santé (probes)

Composant	startupProbe	livenessProbe	readinessProbe
Spring Boot	<code>/actuator/health/liveness</code> , toutes les 5 s, jusqu'à 30 échecs (2 min 30 max)	<code>/actuator/health/liveness</code> , toutes les 10 s, 3 échecs	<code>/actuator/health/readiness</code> , toutes les 5 s, 3 échecs

Composant	startupProbe	livenessProbe	readinessProbe
ml-service	/health/live , jusqu'à 2 min (chargement du modèle)	/health/live	/health/ready (modèle chargé et préchauffé)
postgres	-	pg_isready	pg_isready
rabbitmq	-	rabbitmq-diagnostics ping	rabbitmq-diagnostics check_port_connectivity

Règles de conception :

- **La liveness ne vérifie jamais une dépendance externe.** Si elle vérifiait PostgreSQL, une panne de la base ferait redémarrer en boucle tous les services : une panne en cascade.
- **La readiness vérifie uniquement les dépendances indispensables.**
 - La base de données, oui, pour les services qui en ont besoin.
 - Le ml-service, non, pour risk-service : il dispose d'un repli (REVUE HUMAINE).
 - RabbitMQ, non : l'outbox met les événements en attente.
- **La startupProbe** protège les démarrages lents (JVM, chargement du modèle) sans rendre la liveness trop tolérante ensuite.

6.8 Mises à jour progressives et rollback

Rolling update sans interruption :

Paramètre	Valeur	Effet
maxSurge / maxUnavailable	1 / 0	Un nouveau pod doit être prêt avant qu'un ancien ne soit arrêté
minReadySeconds	10	Le pod doit rester prêt 10 s avant d'être compté comme disponible
progressDeadlineSeconds	120	Au-delà, le déploiement est déclaré en échec : Argo CD passe en Degraded et une alerte se déclenche
revisionHistoryLimit	5	Anciennes versions conservées

Arrêt propre (graceful shutdown) :

- Spring Boot est configuré avec `server.shutdown=graceful`, et Uvicorn termine les requêtes en cours.
- Un `preStop` de 5 s laisse à Kubernetes le temps de retirer le pod des Services **avant** l'arrêt du processus. Sans ce délai, quelques requêtes échouent à chaque déploiement.
- `terminationGracePeriodSeconds` est fixé à 30.

Disponibilité pendant les opérations :

- **PodDisruptionBudget** `minAvailable: 1` pour chaque composant répliqué. Une opération de maintenance (drain d'un nœud) ne peut pas arrêter tous les replicas à la fois.
- **topologySpreadConstraints** répartissent les replicas sur les 2 workers, de sorte que la panne d'un nœud n'emporte pas tous les replicas d'un service.

Rollback.

- Argo CD est configuré en synchronisation automatique avec `selfHeal`. Un `kubectl rollout undo` manuel serait donc **annulé** par Argo CD, qui rétablit l'état décrit dans Git.
- **Le rollback se fait donc par Git** : `git revert`, puis Argo CD redéploie la version précédente par rolling update.
- Pour la démo, on peut aussi utiliser la fonction « History and Rollback » de l'interface Argo CD.

6.9 ConfigMaps et Secrets

ConfigMap	Contenu
<code><service>-config</code> (un par service)	URL des services appelés, délais max, réglages des circuit breakers, profil Spring
<code>risk-config</code>	Paramètres de la politique de décision
<code>ml-service-config</code>	<code>MODEL_URI</code> (version épinglée, D20), <code>MODEL_MODE</code> (<code>real</code> ou <code>mock</code>)
<code>ml-pipeline-config</code>	Mode du pipeline, <code>AUTO_MERGE</code> , seuils des portes de qualité
<code>simulator-config</code>	Débit (transactions/s), délai des rejets de paiement simulés

Secret	Contenu	Utilisé par
<code><service>-db</code> (un par service)	Utilisateur et mot de passe PostgreSQL du schéma	chaque service
<code>rabbitmq-credentials</code>	Identifiants RabbitMQ	services producteurs et consommateurs
<code>jwt-private-key</code>	Clé privée RS256	auth-service uniquement
<code>card-hmac-key</code>	Clé HMAC du numéro de carte	customer-service uniquement
<code>github-token</code>	Jeton limité aux PR du dépôt	ml-pipeline
<code>fintech-tls</code>	Certificat mkcert	Traefik

Les secrets dans un dépôt Git (GitOps). Un Secret Kubernetes n'est qu'encodé en base64, pas chiffré : il ne peut pas être committé tel quel. Décision D22 : **Sealed Secrets**.

- Les secrets sont chiffrés avec la clé publique du cluster, et seuls les fichiers chiffrés sont committés.
- Un contrôleur léger les déchiffre dans le cluster.
- Argo CD peut ainsi déployer **tout** depuis Git.

6.10 Stockage

Volume	Type	Taille	Remarque
Données PostgreSQL	PVC <code>local-path</code>	5 Gi	Lié à <code>worker-1</code>
Données RabbitMQ	PVC <code>local-path</code>	1 Gi	Lié à <code>worker-1</code>
Artefacts MLflow	PVC <code>local-path</code>	5 Gi	Lié à <code>worker-1</code>
Cache des modèles	<code>hostPath</code> sur chaque worker	-	Un volume <code>local-path</code> ne se partage pas entre deux nœuds. Chaque nœud garde donc son propre cache, et un pod ml-service redémarré sur le même nœud recharge sans MLflow

6.11 Ingress et équilibrage de charge

Hôte	Chemin	Destination
<code>fintech.local</code>	<code>/</code>	frontend
<code>fintech.local</code>	<code>/api</code>	api-gateway
<code>mlflow.fintech.local</code>	<code>/</code>	mlflow
<code>grafana.fintech.local</code>	<code>/</code>	grafana

Hôte	Chemin	Destination
argocd.fintech.local	/	argocd-server

- **TLS** terminé par Traefik avec le certificat mkcert.
- **Accès depuis la machine hôte.** Les noms d'hôte sont ajoutés au fichier `hosts` de la machine. Les ports 80 et 443 sont exposés par la configuration kind (`extraPortMappings`).
- **SSE (notifications).** Délai d'inactivité long sur la route `/api/v1/notifications/stream`, sans mise en tampon.

Deux niveaux d'équilibrage de charge :

1. **Traefik** répartit les requêtes entrantes entre les pods du frontend et de la Gateway.
2. **Les Services Kubernetes** (ClusterIP) répartissent les appels internes entre les pods de chaque service.

Point technique important pour les tests de charge. Un Service Kubernetes répartit les **connexions**, pas les requêtes. La Gateway et risk-service réutilisent leurs connexions HTTP (keep-alive). Après une montée en charge, les **nouveaux pods risquent donc de ne presque rien recevoir**, puisque les connexions existantes restent ouvertes vers les anciens pods.

Correction. Une **durée de vie maximale de 30 s** est fixée pour les connexions des pools HTTP (Gateway, transaction → risk, risk → ml-service). Les connexions sont renouvelées régulièrement et les nouveaux pods reçoivent du trafic en quelques secondes. Cet effet sera mis en évidence dans Grafana lors des expériences (section 9).

6.12 NetworkPolicies

Par défaut, tout trafic entrant est refusé dans les namespaces `fintech`, `ml` et `data`. Seuls les flux suivants sont autorisés :

Source	Destination	Port
traefik	frontend, api-gateway, mlflow, grafana, argocd-server	HTTP
api-gateway	auth, customer, transaction, risk, notification, audit, ml-service (<code>/api/v1/ml/**</code>)	HTTP
transaction-service	risk-service	HTTP
risk-service	customer-service, ml-service	HTTP
Services Spring Boot, mlflow	postgres	5432
Services Spring Boot	rabbitmq	5672
ml-service, ml-pipeline	mlflow	HTTP
monitoring (Prometheus)	tous les pods	port des métriques

Conséquence. Même si un attaquant obtenait un pod dans le cluster, il ne pourrait pas appeler directement `/predict` ou `/internal/risk/assess` depuis un pod non autorisé.

6.13 Organisation des manifests

- **Nos services : Kustomize**, natif dans `kubectl` et Argo CD. Une base commune par service et une surcouche (`overlay`) `demo`.
- **Composants tiers : charts Helm officiels** de Traefik, kube-prometheus-stack, Argo CD et Sealed Secrets, pilotés comme applications Argo CD.

- **PostgreSQL et RabbitMQ** : simples StatefulSets écrits par l'équipe, avec les **images officielles**. C'est plus lisible et sans dépendance à des charts tiers.
- **Argo CD « app of apps »** : une application racine déclare toutes les autres. L'installation complète se fait en une commande.

```

deploy/
├─ bootstrap/           # config kind, installation d'Argo CD, script make cluster
├─ argocd/              # application racine + une Application par composant
├─ base/
│   ├─ api-gateway/     # deployment, service, hpa, pdb, configmap, networkpolicy
│   └─ risk-service/
│       └─ ...
├─ overlays/demo/       # tags d'images, replicas, paramètres de la démo
├─ platform/            # valeurs Helm : traefik, monitoring, sealed-secrets
└─ secrets/             # SealedSecrets chiffrés uniquement

```

Mise en route. Une nouvelle machine peut recréer tout l'environnement en deux commandes : `make cluster` crée kind et installe Argo CD, puis Argo CD installe tout le reste.

6.14 Livrables et calendrier

Livable	Semaine
Docker Compose de développement + cluster kind + squelette CI	S1-S3
Dockerfiles des 10 images, analyse Trivy	S4-S6
Manifests de base (Deployments, Services, Ingress, probes, ressources, ConfigMaps, Secrets)	S6-S8
Argo CD + app of apps + Sealed Secrets	S7-S8
HPA, PDB, répartition sur les nœuds, NetworkPolicies	S8-S9
Démonstration du rolling update et du rollback	S9-S10

7. CI/CD et DevSecOps

Responsable : P4 (DevOps). **Support :** P1 (tests d'intégration et de bout en bout), P3 (CI du code ML), tous (qualité du code).

Le sujet impose la chaîne **Git → Build → Tests → SonarQube → Security Scan → Docker → Kubernetes → Integration Tests**, avec déploiement et rollback automatiques. Elle est répartie entre **GitHub Actions** (intégration continue) et **Argo CD** (déploiement continu, décision D8).

7.1 Organisation du dépôt (monorepo)

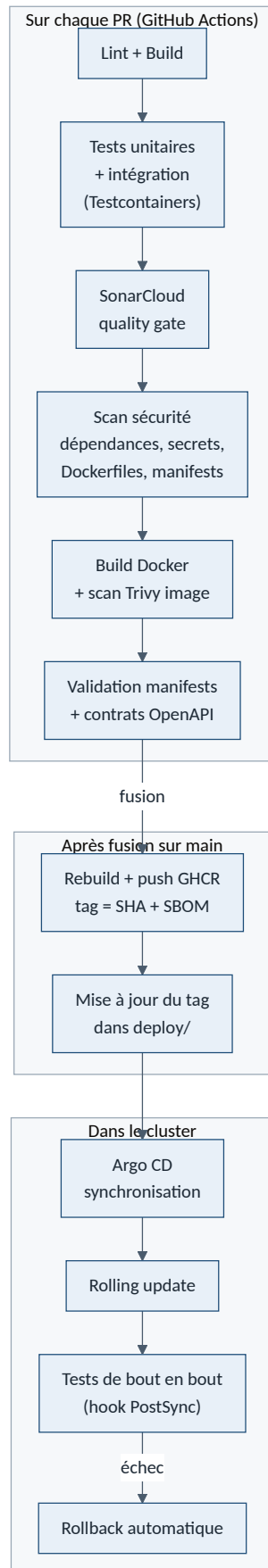
```
fintech-mlops/
├── services/                # 7 services Spring Boot (P1)
│   ├── api-gateway/
│   ├── auth-service/
│   └── ...
├── ml/                      # code ML : prétraitement, benchmark, stats, pipeline (P2, P3)
├── ml-service/              # FastAPI (P3)
├── frontend/                # React (P5)
├── simulator/               # rejeu Sparkov + rejets de paiement simulés
├── deploy/                  # Kustomize, Argo CD, Helm values, SealedSecrets (P4)
├── monitoring/              # dashboards Grafana (JSON), règles d'alerte (P5)
├── load-tests/              # scénarios k6 et chaos (P4)
├── e2e-tests/               # tests de bout en bout post-déploiement (P1)
├── contracts/               # OpenAPI de tous les services + ml-service
├── docs/                    # architecture, figures, captures
├── .github/workflows/       # pipelines CI/CD
├── CODEOWNERS
└── README.md
```

CODEOWNERS. Chaque dossier a un responsable : une PR qui touche `ml-service/` demande automatiquement la revue de P3, par exemple.

7.2 Workflow Git

Règle	Mise en œuvre
Branches	Trunk-based : <code>main</code> + branches courtes <code>feat/...</code> , <code>fix/...</code> (quelques jours au plus)
Protection de <code>main</code>	Ruleset GitHub : PR obligatoire, 1 approbation , vérifications CI obligatoires, historique linéaire
Commits	Convention <i>Conventional Commits</i> (<code>feat:</code> , <code>fix:</code> , <code>ci:</code> ...) pour un historique lisible
Taille des PR	Petites et ciblées : une PR = un changement cohérent
Exception	Seul le bot de la CI peut écrire directement sur <code>main</code> , et uniquement dans le fichier des tags d'images (7.6). Le pipeline ML, lui, passe par une PR soumise aux mêmes vérifications (section 5.5)

7.3 Vue d'ensemble de la chaîne



Correspondance avec le sujet :

Étape du sujet	Où	Outil
Git	GitHub, ruleset sur <code>main</code>	GitHub
Build	PR et <code>main</code>	Maven, pip, npm (Vite)
Tests	PR	JUnit + Testcontainers, pytest, Vitest
SonarQube	PR	SonarCloud (décision D10)
Security Scan	PR et <code>main</code>	Trivy (dépendances, images, configuration), gitleaks, Dependabot
Docker	<code>main</code>	Docker Buildx, GHCR
Kubernetes	Cluster	Argo CD, Kustomize
Integration Tests	CI (Testcontainers) et cluster (bout en bout, PostSync)	JUnit, pytest
Rollback automatique	Cluster puis GitHub	Hook PostSync + workflow <code>rollback.yml</code>

7.4 Pipeline d'une PR

Exécution sélective. Des filtres de chemins déclenchent uniquement les jobs concernés : une PR qui ne modifie que `frontend/` ne reconstruit pas les 7 services Java. Les 7 services partagent un **workflow réutilisable** lancé en matrice.

Job	Composants	Contenu	Bloquant
<code>java-ci</code>	Services Spring Boot	Compilation, tests unitaires, tests d'intégration Testcontainers (PostgreSQL, RabbitMQ), couverture JaCoCo	oui
<code>python-ci</code>	ml, ml-service, simulator	Ruff (lint), pytest, couverture	oui
<code>ml-smoke</code>	ml	Pipeline complet sur 1 % des données avec un MLflow local au runner (section 5.6)	oui
<code>frontend-ci</code>	frontend	ESLint, Vitest, build Vite	oui
<code>sonar</code>	tous les composants modifiés	Analyse SonarCloud + attente du quality gate	oui
<code>security</code>	tout le dépôt	gitleaks (secrets), Trivy <code>fs</code> (vulnérabilités des dépendances), Trivy <code>config</code> (Dockerfiles, manifests)	oui (critique / élevé)
<code>docker</code>	images modifiées	Build (sans publication) + scan Trivy de l'image	oui (critique corrigeable)
<code>manifests</code>	deploy/	<code>kustomize build</code> + <code>kubeconform</code> (validation des schémas Kubernetes)	oui
<code>contracts</code>	contracts/	Comparaison du contrat OpenAPI généré par le code avec le contrat versionné	oui

Cibles de temps :

- une PR ordinaire (un seul composant) doit s'exécuter en **moins de 10 min** ;
- une PR qui touche tout le dépôt, en **moins de 20 min**.

Les caches Maven, pip, npm et des couches Docker (cache GitHub Actions) sont activés.

7.5 Contrôles DevSecOps

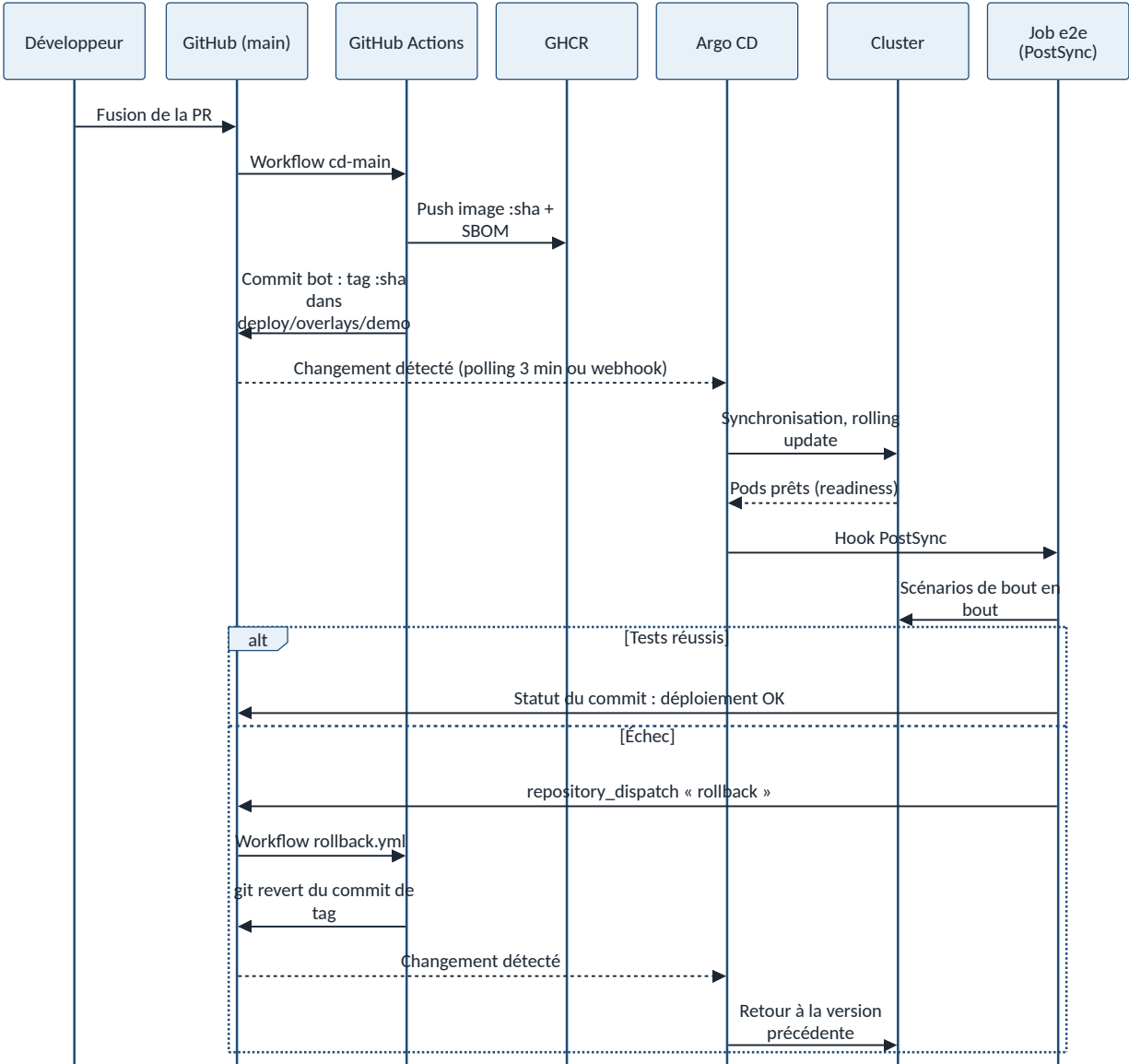
Contrôle	Outil	Seuil	Moment
Analyse statique du code (SAST)	SonarCloud	Quality gate : 0 nouveau bug, 0 nouvelle vulnérabilité, hotspots de sécurité revus, couverture du nouveau code $\geq 60\%$, duplication $< 3\%$	Chaque PR
Vulnérabilités des dépendances (SCA)	Trivy <code>fs</code> + Dependabot	Échec si CVE critique ou élevée corrigable	Chaque PR + mises à jour Dependabot hebdomadaires
Secrets dans le code	gitleaks + protection push GitHub	Échec au moindre secret détecté	Chaque PR + chaque push
Vulnérabilités des images	Trivy <code>image</code>	Échec si CVE critique corrigable	Chaque build d'image
Mauvaises configurations	Trivy <code>config</code>	Échec si élevée : conteneur root, absence de limits, privilèges, etc.	Chaque PR touchant <code>deploy/</code> ou un Dockerfile
Inventaire logiciel (SBOM)	Trivy (format CycloneDX)	-	Chaque image publiée, jointe au run
Moindre privilège de la CI	Permissions GitHub Actions	<code>GITHUB_TOKEN</code> en lecture seule par défaut, écriture accordée job par job	Toujours
Rescan nocturne	Trivy sur les images en production	Nouvelle CVE publiée → issue GitHub créée automatiquement	Chaque nuit

Seulement les vulnérabilités « corrigables ». Bloquer sur une vulnérabilité sans correctif disponible arrêterait l'équipe sans action possible. Ces cas sont listés dans un fichier `.trivyignore` commenté et daté, relu chaque semaine.

Bonus possibles (décision D25 : ajoutés seulement s'il reste du temps après S12) :

- signature des images avec cosign ;
- scan dynamique (DAST) avec OWASP ZAP en mode baseline sur l'application déployée.

7.6 Déploiement continu



Mise à jour des tags d'images.

- Le workflow `cd-main` modifie **uniquement** `deploy/overlays/demo/kustomization.yaml`. Une vérification refuse le commit si un autre fichier est touché.
- Ce commit utilise une identité de CI autorisée à contourner la protection de `main`, uniquement pour ce fichier. **P4 valide ce mécanisme dès S2.**
- Le commit porte `[skip ci]` pour ne pas relancer la CI en boucle.

Accès d'Argo CD à Git. Argo CD interroge GitHub toutes les 3 minutes. C'est le cluster qui sort vers GitHub : aucune ouverture du cluster vers Internet n'est nécessaire. Pour la démo, un « Refresh » manuel dans l'interface accélère la synchronisation.

7.7 Tests de bout en bout et rollback automatique

Tests exécutés après chaque déploiement par un Job Kubernetes (hook PostSync d'Argo CD, image `e2e-tests:sha`) :

#	Scénario	Vérification
1	Santé	Tous les Deployments sont disponibles, et <code>/actuator/health</code> et <code>/health/ready</code> sont OK

#	Scénario	Vérification
2	Authentification	Connexion ANALYSTE → JWT valide. Un accès sans jeton est refusé (401)
3	Transaction complète	POST /api/v1/transactions → réponse avec une décision parmi APPROUVEE , EN_REVUE , BLOQUEE et la version du modèle
4	Version du modèle	/api/v1/ml/model renvoie la version attendue (celle de la ConfigMap)
5	Chaîne asynchrone	L'événement de la transaction du scénario 3 apparaît dans l'audit en moins de 15 s (outbox → RabbitMQ → audit)
6	Revue humaine	Une transaction mise en revue peut être traitée par l'analyste, et son statut final est mis à jour
7	Sécurité réseau	/predict n'est pas accessible via l'Ingress (404)

Mode mock pour les tests. Le scénario 6 a besoin d'une transaction qui part à coup sûr en revue. Le ml-service mock rend cela déterministe dans la CI. Dans le cluster, le Job choisit une transaction de référence connue pour être incertaine, identifiée lors de l'évaluation du modèle.

Rollback automatique.

1. En cas d'échec, le Job e2e envoie un événement `repository_dispatch` à GitHub, avec le jeton limité déjà utilisé par le pipeline ML.
2. Le workflow `rollback.yml` fait un `git revert` du dernier commit de tag.
3. Argo CD redéploie automatiquement la version précédente.
4. Une issue GitHub est créée avec les logs du test en échec.

Tout se fait **par Git**, ce qui est cohérent avec la décision D20 et avec le fonctionnement de `selfHeal` (section 6.8).

Rollback manuel. Un `git revert` de n'importe quel commit de déploiement, ou « History and Rollback » dans l'interface Argo CD.

7.8 Workflows GitHub Actions

Fichier	Déclencheur	Rôle
<code>ci-java.yml</code>	PR (réutilisable, matrice de services)	Build, tests, couverture
<code>ci-python.yml</code>	PR	ml, ml-service, simulator
<code>ci-frontend.yml</code>	PR	React
<code>ml-smoke.yml</code>	PR touchant <code>ml/</code>	Pipeline de fumée
<code>quality-security.yml</code>	PR	SonarCloud, Trivy, gitleaks
<code>manifests.yml</code>	PR touchant <code>deploy/</code>	Kustomize + kubeconform
<code>cd-main.yml</code>	Push sur <code>main</code>	Build, push, SBOM, mise à jour des tags
<code>ml-pipeline-image.yml</code>	Push sur <code>main</code> touchant <code>ml/</code>	Image du Job de pipeline ML
<code>rollback.yml</code>	<code>repository_dispatch</code>	Revert automatique
<code>nightly.yml</code>	Planifié (chaque nuit)	Rescan Trivy des images en production

7.9 Limite documentée : un seul environnement

Il n'y a pas d'environnement de préproduction séparé : un second cluster ne tiendrait pas dans 16 Go. Trois mécanismes compensent :

- les **tests d'intégration** Testcontainers en CI, avant toute fusion ;

- le **rolling update** protégé par les readiness probes, qui bloque une version incapable de démarrer ;
- les **tests de bout en bout après déploiement**, avec rollback automatique.

7.10 Livrables et calendrier

Livrable	Responsable	Semaine
Dépôt, ruleset, CODEOWNERS, squelette CI (build + tests)	P4 (avec P1)	S1-S3
SonarCloud + Trivy + gitleaks dès les premiers services (correctif 3 de la revue)	P4	S4-S6
Build et publication des images, validation des manifests	P4	S6-S7
CD avec Argo CD, mise à jour automatique des tags	P4	S7-S8
Tests de bout en bout (Job PostSync)	P1 (tests), P4 (intégration)	S9-S10
Rollback automatique + démonstration	P4	S10

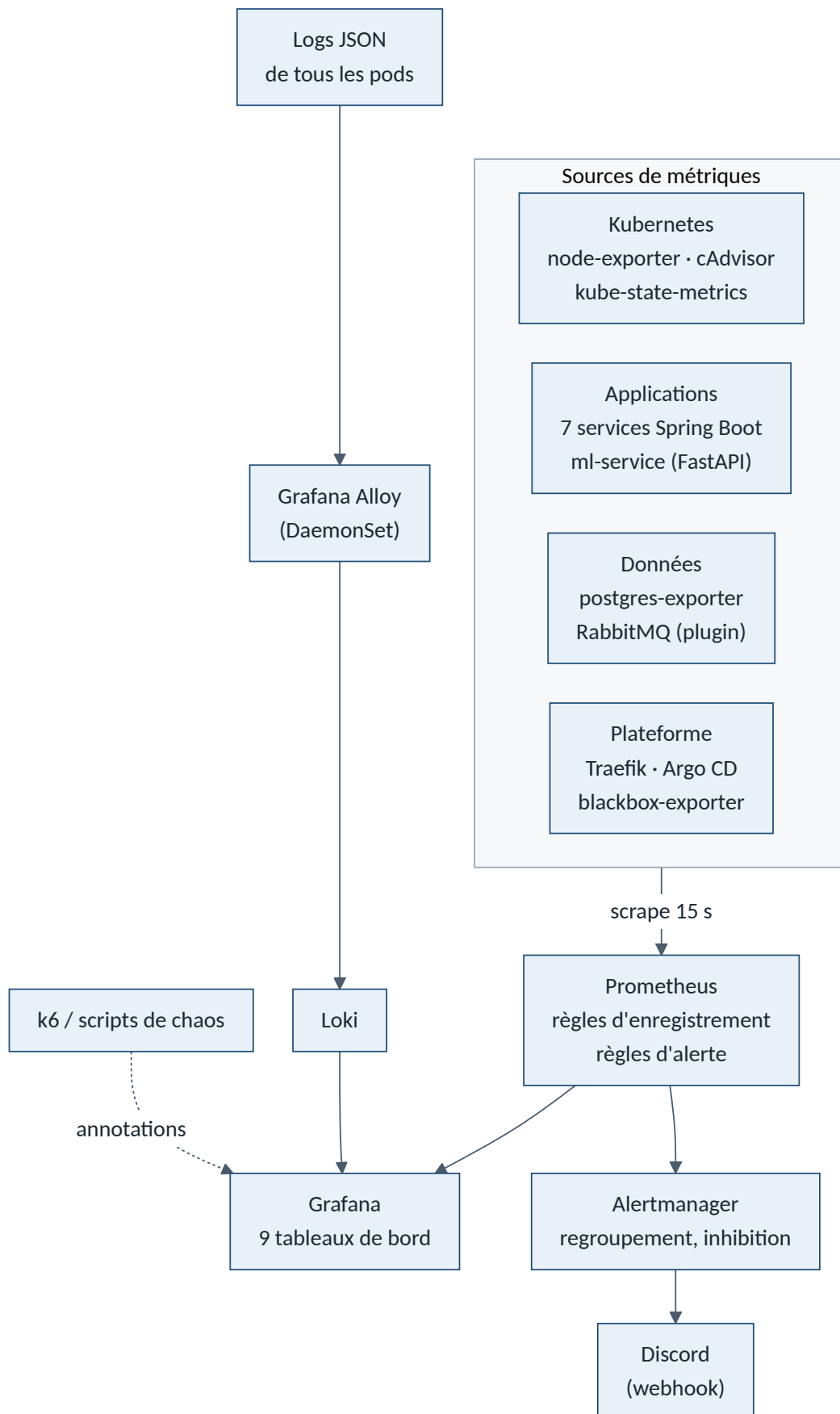
8. Observabilité : supervision, tableaux de bord et alertes

Responsable : P5 (Frontend et observabilité). **Support** : P1 et P3 (métriques exposées), P4 (installation dans le cluster).

Objectif. Démontrer une **supervision continue** : à tout moment, on sait si la plateforme est en bonne santé, pourquoi elle ralentit, si elle passe à l'échelle et si le modèle se comporte bien. Un problème déclenche une alerte **avant** qu'un utilisateur ne le remarque.

8.1 Architecture de supervision

La pile est installée avec le chart Helm **kube-prometheus-stack** (namespace `monitoring`). Il regroupe Prometheus Operator, Prometheus, Alertmanager, Grafana, node-exporter et kube-state-metrics. **Loki** et **Grafana Alloy** complètent la pile pour les logs (section 8.6).



Découverte automatique des cibles. Chaque composant déclare un **ServiceMonitor** (ressource du Prometheus Operator), versionné avec ses manifests. Un nouveau service est supervisé dès son déploiement, sans toucher à la configuration de Prometheus.

Réglages :

Paramètre	Valeur	Justification
Intervalle de collecte	15 s (5 s pour kube-state-metrics et la Gateway pendant les expériences)	Compromis charge / précision. Les temps de mise à l'échelle sont mesurés plus finement pendant les expériences
Rétention	7 jours, 5 Gi	Suffisant pour couvrir les expériences et la démo
Ressources	Prometheus : 512 Mi à 1 Gi ; Grafana : 128 Mi ; Alertmanager : 64 Mi ; Loki + Alloy : ~0,6 Gi	Dans le budget de la section 6.5

8.2 Couverture des sources demandées par le sujet

Exigence du sujet	Source	Exemples de métriques
Nœuds et pods Kubernetes	node-exporter, cAdvisor, kube-state-metrics	<code>node_cpu_seconds_total</code> , <code>kube_pod_status_ready</code> , <code>kube_pod_container_status_restarts_total</code>
Microservices Spring Boot	Micrometer	<code>http_server_requests_seconds_*</code> , <code>jvm_memory_used_bytes</code> , <code>hikaricp_connections_active</code> , <code>resilience4j_circuitbreaker_state</code>
FastAPI	prometheus-fastapi-instrumentator + métriques ML (section 5.8)	<code>http_request_duration_seconds_*</code> , <code>ml_inference_duration_seconds_*</code>
Bases de données	postgres-exporter	Connexions, transactions/s, taille des schémas, requêtes lentes
CPU, mémoire, réseau	cAdvisor, node-exporter	<code>container_cpu_usage_seconds_total</code> , <code>container_memory_working_set_bytes</code> , <code>container_network_*</code>
Débit des requêtes	Micrometer, Traefik	<code>rate(http_server_requests_seconds_count[1m])</code>
Latence des réponses	Histogrammes Micrometer et FastAPI	p50, p95, p99 via <code>histogram_quantile</code>
Erreurs HTTP	Micrometer, Traefik	Proportion de réponses <code>5xx</code> et <code>4xx</code>
Nombre de replicas	kube-state-metrics	<code>kube_deployment_status_replicas_available</code>
Activité de l'HPA	kube-state-metrics	<code>kube_horizontalpodautoscaler_status_current_replicas</code> , <code>..._desired_replicas</code>
Temps d'inférence ML	ml-service	<code>ml_inference_duration_seconds</code>

Métriques supplémentaires de la plateforme :

- RabbitMQ : profondeur des queues, messages en DLQ, débit de consommation ;
- outbox : `outbox_pending` ;
- état des circuit breakers ;
- performance du modèle en production : rejets de paiement simulés, D19 ;
- état des applications Argo CD ;
- disponibilité vue de l'extérieur (sondes blackbox).

Conventions de métriques :

- **Étiquettes communes.** Tous les services Spring Boot ajoutent l'étiquette `application=<nom-du-service>` .

- **Histogrammes de latence.** Les **histogrammes de latence** sont activés avec des bornes alignées sur les objectifs : 0,05 · 0,1 · 0,25 · 0,5 · 1 · 2 · 5 s. Le p95 est calculable et le seuil de 2 s du sujet correspond exactement à une borne.

8.3 Règles d'enregistrement

Des **recording rules** précalculent les expressions coûteuses ou utilisées partout, pour des tableaux de bord rapides et des alertes lisibles :

Règle	Expression (résumée)
<code>app:http_requests:rate1m</code>	Débit par service
<code>app:http_errors:ratio1m</code>	Part des réponses 5xx par service
<code>app:http_latency:p95_1m</code>	p95 de latence par service
<code>ml:review_rate:10m</code>	Part des ensembles conformes {0,1} ou vides
<code>ml:inference_latency:p95_1m</code>	p95 du temps d'inférence
<code>risk:conformal_coverage:30m</code>	Couverture réelle mesurée grâce aux rejets de paiement simulés, par classe
<code>risk:precision:30m</code> , <code>risk:recall:30m</code>	Performance réelle du modèle en production

8.4 Tableaux de bord Grafana

Dashboards as code. Les 9 tableaux de bord demandés sont versionnés en JSON dans `monitoring/dashboards/`. Ils sont chargés automatiquement dans Grafana via des ConfigMaps (mécanisme de provisioning de kube-prometheus-stack) et donc déployés par Argo CD comme le reste.

#	Tableau de bord	Question à laquelle il répond	Panneaux principaux
1	Santé du système	La plateforme fonctionne-t-elle ?	Feu tricolore par service (up, prêt, erreurs), disponibilité externe (blackbox), alertes actives, version déployée de chaque service et du modèle
2	Performance des microservices	Quel service est lent ou en erreur ?	Méthode RED par service : débit, erreurs, latence p50/p95/p99. JVM, pool de connexions, circuit breakers
3	Cluster Kubernetes	Les ressources suffisent-elles ?	CPU et mémoire par nœud et par namespace, pods par état, redémarrages, utilisation par rapport aux requests et limits
4	Charge et scalabilité	La plateforme suit-elle la charge ?	Débit entrant, latence p95, replicas actuels et souhaités par l'HPA, CPU par rapport à la cible de 70 %, répartition du trafic entre pods
5	Haute disponibilité	Que se passe-t-il pendant une panne ?	Pods prêts par service, redémarrages, disponibilité externe, erreurs pendant l'incident, temps de récupération , file outbox et queues RabbitMQ
6	Inférence ML	Le modèle répond-il assez vite ?	Temps d'inférence p50/p95, débit de prédictions, erreurs de validation, version en production, état du circuit breaker risk → ml
7	Performance du modèle	Le modèle est-il toujours bon ?	Précision, rappel et taux de faux positifs en production , matrice de confusion glissante, comparaison avec les valeurs du test final (lignes de référence), distribution des probabilités
8	Incertitude des prédictions	Les incertitudes sont-elles sous contrôle ?	Taux d'automatisation et de revue, couverture conforme réelle par rapport à 1 - α , distribution de l'incertitude, file de revue humaine (taille, ancienneté), accord entre analystes et vérité
9	Erreurs et alertes	Qu'est-ce qui ne va pas, et depuis quand ?	Alertes actives et historique, top des erreurs par service et par route, DLQ, échecs de déploiement (Argo CD), logs d'erreur récents (Loki)

Annotations. Les événements marquants apparaissent comme des lignes verticales sur tous les graphiques :

- **déploiements** (Argo CD) ;
- **début et fin de chaque phase d'expérience** : charge normale, élevée, surcharge, récupération ;
- **injections de pannes** (suppression de pod, arrêt d'un nœud).

Les scripts k6 et de chaos créent ces annotations via l'API Grafana. On voit ainsi immédiatement la cause et l'effet dans les captures du README et dans la démo.

Répartition avec React. Grafana sert la **supervision technique** (équipe d'exploitation). Le tableau de bord React sert le **métier** (analystes) : il affiche la performance et l'incertitude du modèle à partir des API du ml-service et de risk-service. Grafana n'est pas intégré dans React : cela exigerait un accès anonyme à Grafana, ce qui affaiblirait la sécurité.

8.5 Alertes

Alertes exigées par le sujet :

Alerte du sujet	Règle (PromQL simplifiée)	Durée	Gravité
CPU > 80 %	CPU utilisé / limite CPU du conteneur > 0,8	2 min	warning
Temps de réponse > 2 s	<code>app:http_latency:p95_1m{application="api-gateway"} > 2</code>	2 min	warning
Taux d'erreur > 5 %	<code>app:http_errors:ratio1m{application="api-gateway"} > 0.05</code>	2 min	critical
Incertitude ML élevée → revue humaine	<code>ml:review_rate:10m > 0.05</code> (normal ≤ 2 %)	10 min	warning
Panne de pod	Redémarrage d'un conteneur sur les 5 dernières min, ou pod non prêt	1 min	warning
Service indisponible	<code>kube_deployment_status_replicas_available == 0</code> ou sonde blackbox en échec	1 min	critical

Pourquoi 80 % alors que l'HPA vise 70 %. L'HPA agit à partir de 70 %. L'alerte à 80 % ne se déclenche donc que si l'autoscaling **ne suffit pas** : maximum atteint ou nouveaux pods trop lents à démarrer. C'est exactement la situation qui mérite l'attention d'un humain.

« **Incertitude élevée → revue humaine** » fonctionne à deux niveaux :

1. **Par transaction, automatiquement.** risk-service envoie chaque transaction incertaine en revue humaine (section 2.5). Aucune alerte n'est nécessaire, c'est le fonctionnement normal.
2. **Globalement, par une alerte.** Si la **part** des transactions incertaines dépasse 5 %, alors que 2 % sont attendus, le modèle rencontre des données inhabituelles (dérive probable) et la file des analystes va déborder. L'alerte prévient l'équipe.

Alertes supplémentaires de la plateforme :

Alerte	Condition	Gravité
HPA au maximum	Replicas actuels = maximum, pendant 5 min	warning
Circuit breaker ouvert	risk → ml ou transaction → risk ouvert	warning
Outbox en retard	<code>outbox_pending > 100</code> pendant 2 min (RabbitMQ en difficulté)	warning
Messages en DLQ	Au moins 1 message dans une DLQ	warning
Couverture conforme en baisse	<code>risk:conformal_coverage:30m < (1 - α) - 0,05</code>	critical
Modèle non chargé	Aucun pod ml-service prêt	critical
Latence d'inférence	p95 d'inférence > 50 ms pendant 5 min	warning

Alerte	Condition	Gravité
Déploiement en échec	Application Argo CD Degraded	critical
Nœud indisponible	Nœud NotReady pendant 1 min	critical
Disque	PVC utilisé à plus de 80 %	warning

Réduction du bruit (Alertmanager) :

- **Regroupement** par service et par gravité, pour recevoir une seule notification par incident.
- **Inhibition.** Quand « Service indisponible » est active pour un service, ses alertes de latence et d'erreurs sont masquées : la cause est déjà connue.
- **Répétition.** Toutes les 4 h pour une alerte **warning** non résolue, toutes les 30 min pour une **critical**. Une notification est envoyée à la **résolution**.

Canal de notification : Discord (décision D26).

- Alertmanager envoie les alertes vers un **salon Discord** de l'équipe grâce à son récepteur Discord intégré.
- L'URL du webhook est stockée dans un SealedSecret.
- Deux salons distincts, **#alertes-critiques** et **#alertes-warning**, pour que les alertes critiques ne soient pas noyées.
- Chaque message contient : nom de l'alerte, service, valeur mesurée, lien vers le tableau de bord Grafana et vers le runbook.
- **Pendant la démo**, les alertes arrivent en direct sur les téléphones de l'équipe.

Chaque alerte a une **fiche d'intervention (runbook)**. Il s'agit d'un court fichier Markdown dans **monitoring/runbooks/**, lié depuis l'alerte : signification, tableau de bord à ouvrir, premières actions.

8.6 Logs centralisés (Loki, décision D27)

Collecte.

- Les services écrivent des **logs JSON** sur la sortie standard, avec **correlationId**, nom du service et niveau (section 3.10).
- **Grafana Alloy**, déployé en DaemonSet (un pod par nœud), lit les logs de tous les conteneurs et les envoie à **Loki**. Alloy est l'agent recommandé par Grafana, qui remplace Promtail, désormais obsolète.
- Loki fonctionne en **mode monolithique** (un seul binaire), avec stockage sur disque (PVC 5 Gi) et **rétenion de 3 jours**. C'est le mode le plus léger, suffisant pour un cluster local.

Usages :

Usage	Mise en œuvre
Suivre une transaction de bout en bout	Recherche par correlationId dans Grafana Explore : on voit les logs de la Gateway, de transaction, de risk, du ml-service et de l'audit pour une même transaction
Passer d'une métrique aux logs	Depuis un pic d'erreurs sur un graphique, on ouvre les logs du même service sur la même période en un clic (Explore, vue partagée)
Logs dans les tableaux de bord	Panneau « erreurs récentes » dans les tableaux de bord 2 (microservices), 5 (haute disponibilité) et 9 (erreurs et alertes)
Alertes sur les logs	Règle Loki : apparition de OutOfMemoryError ou d'une erreur de chargement du modèle → alerte critical

Étiquettes Loki. Uniquement `namespace` , `app` , `pod` et `level` , pour garder un index petit. Le `correlationId` est lu dans le contenu JSON au moment de la recherche, et non indexé : un index par transaction exploserait la mémoire de Loki.

Ressources. Loki : 256 à 512 Mi. Alloy : ~100 Mi par nœud. Environ **0,6 Go** au total, intégrés au budget de la section 6.5.

Hors périmètre. Traçage distribué (Tempo, OpenTelemetry). Le `correlationId` dans les logs couvre l'essentiel du besoin pour ce projet.

8.7 Accès et sécurité

- **Accès.** Grafana est accessible via `https://grafana.fintech.local` (Traefik, TLS).
- **Secrets.** Le mot de passe administrateur est stocké dans un SealedSecret (D22).
- **Rôles.** Deux comptes : `admin` (P5, P4) et un compte **lecture seule** pour la démo et le reste de l'équipe.
- **Prometheus et Alertmanager ne sont pas exposés** à l'extérieur du cluster. On y accède par `kubectll port-forward` si nécessaire.
- **Seule exception : le récepteur *remote write* de Prometheus.** Pendant les expériences, il est exposé avec mot de passe pour recevoir les métriques de k6 depuis le portable de charge (section 9.2).

8.8 Livrables et calendrier

Livable	Semaine
Prometheus + Grafana dans Docker Compose, branchés sur les premiers services (correctif du partenaire : ne pas attendre Kubernetes)	S4-S6
kube-prometheus-stack dans le cluster, ServiceMonitors, exporters, sondes blackbox	S8
Règles d'enregistrement + tableaux de bord 1 à 6	S9-S10
Tableaux de bord 7 et 8 (après la mise en place des rejets de paiement simulés)	S10-S11
Loki + Alloy, recherche par <code>correlationId</code> , panneaux de logs	S9-S10
Règles d'alerte, Alertmanager + Discord, runbooks	S10-S11
Validation pendant les expériences : chaque alerte déclenchée au moins une fois et capturée	S12

9. Stratégie de tests de charge et de haute disponibilité

Responsable : P4 (DevOps). **Support :** P5 (observation dans Grafana, captures), P1 (comportement des services).

Principe. Les expériences sont conduites **comme des expériences scientifiques**, dans le même esprit que la partie ML :

- des **hypothèses** sont formulées à l'avance, avec des critères de réussite chiffrés (section 1.4) ;
- chaque expérience suit un **protocole** écrit et reproductible ;
- chaque expérience est **répétée 3 fois**, et on rapporte la médiane et l'étendue ;
- les résultats, favorables ou non, sont tous publiés dans le README.

9.1 Hypothèses à vérifier

#	Hypothèse	Critère de réussite
H1	En charge normale, la plateforme respecte ses objectifs sans autoscaling	p95 < 500 ms, erreurs < 1 %, replicas au minimum
H2	En charge élevée, l'HPA ajoute des replicas et la latence reste acceptable	Mise à l'échelle < 2 min, p95 < 2 s, erreurs < 1 %
H3	En surcharge, la plateforme se dégrade de façon contrôlée : elle refuse vite au lieu de s'effondrer	Aucun pod tué par manque de mémoire (OOMKilled), aucune transaction approuvée sans décision, alertes déclenchées
H4	Après la surcharge, la plateforme revient à la normale sans intervention humaine	Retour à p95 < 500 ms et erreurs < 1 % en moins de 3 min
H5	La perte d'un pod sur le chemin critique est invisible pour l'utilisateur	Erreurs < 0,5 % pendant l'incident, pod remplacé en moins de 60 s
H6	La perte d'un nœud complet n'interrompt pas le service	Service disponible pendant toute la panne, pods remplacés en moins de 2 min
H7	Une panne de RabbitMQ ne bloque pas les paiements et ne perd aucun événement	Transactions traitées normalement, 0 événement perdu après le retour du broker
H8	Une panne totale du ml-service ne conduit jamais à une approbation automatique	100 % des transactions en REVUE pendant la panne, circuit breaker ouvert, retour automatique ensuite
H9	Une panne de PostgreSQL provoque une interruption courte, sans perte de données	Temps de récupération mesuré, 0 transaction perdue
H10	Un déploiement défectueux n'affecte pas le service	0 erreur utilisateur, rolling update bloqué, rollback automatique (section 7.7)

9.2 Outils

Outil	Rôle
k6	Génération de charge. Scripts en JavaScript, seuils de réussite intégrés, envoi de ses métriques directement dans Prometheus (remote write). Les courbes côté client et côté serveur apparaissent donc dans les mêmes tableaux de bord Grafana
Scripts de chaos (bash + kubect! + docker)	Injection de pannes : suppression de pods, mise à 0 d'un Deployment, arrêt d'un nœud kind
make experiment	Orchestration d'une expérience complète : annotation de début dans Grafana, charge k6, injection de panne à un instant précis, annotation de fin, export des résultats
Grafana	Observation, captures, enregistrement vidéo pour la démo

Machine de génération de charge (décision D28). k6 s'exécute sur le **portable d'un autre membre**, connecté au même réseau local que la machine du cluster. Si k6 tournait sur la même machine, il se partagerait le processeur avec la plateforme et fausserait les mesures en surcharge.

Mise en place :

- **Accès à l'Ingress.** Le fichier `hosts` du portable de charge fait pointer `fintech.local` vers l'adresse IP locale de la machine du cluster. Les ports 80 et 443 de kind écoutent sur toutes les interfaces, et le pare-feu de la machine du cluster autorise ces ports depuis le réseau local.
- **Métriques de k6.** Le récepteur `remote write` de Prometheus est exposé via Traefik (`prometheus.fintech.local`), protégé par mot de passe et **activé uniquement pendant les expériences**.
- **Qualité du réseau.** Un **câble Ethernet** est préférable au Wi-Fi. La latence réseau de base (ping) est mesurée au début de chaque session et rapportée dans les résultats, pour distinguer la latence du réseau de celle de la plateforme.

Pourquoi des scripts plutôt que Chaos Mesh. Les pannes à simuler sont simples (supprimer, arrêter). Des scripts suffisent, sont lisibles, et ne consomment pas de RAM dans le cluster.

9.3 Modèle de charge

Charge « ouverte ». k6 utilise l'exécuteur `ramping-arrival-rate` : il envoie **un nombre fixe de requêtes par seconde**, quelle que soit la vitesse des réponses.

L'autre approche serait un nombre fixe d'utilisateurs virtuels : chacun attend sa réponse avant d'envoyer la suivante. Quand le système ralentit, les utilisateurs envoient donc moins de requêtes, et la surcharge **se cache d'elle-même** (biais appelé *coordinated omission*). La charge ouverte reproduit la réalité : les paiements continuent d'arriver même si le système ralentit.

Données envoyées :

- Chaque requête est une vraie transaction du jeu de test Sparkov, chargée une seule fois en mémoire par k6.
- Chaque requête reçoit un `trans_num` **unique**. Sinon, l'idempotence de transaction-service (section 3.4) renverrait la réponse déjà enregistrée sans rien recalculer, et le test mesurerait un cache au lieu de la plateforme.
- Le jeton JWT (rôle SIMULATEUR) est obtenu une seule fois au démarrage du test.

Trajet des requêtes. Toutes les requêtes passent par le trajet réel : Traefik → Gateway → transaction → risk → customer / ml-service → PostgreSQL → RabbitMQ.

9.4 Calibration : test de capacité

Les niveaux de charge ne sont pas choisis au hasard. On mesure d'abord la **capacité réelle** de la plateforme sur la machine de démonstration :

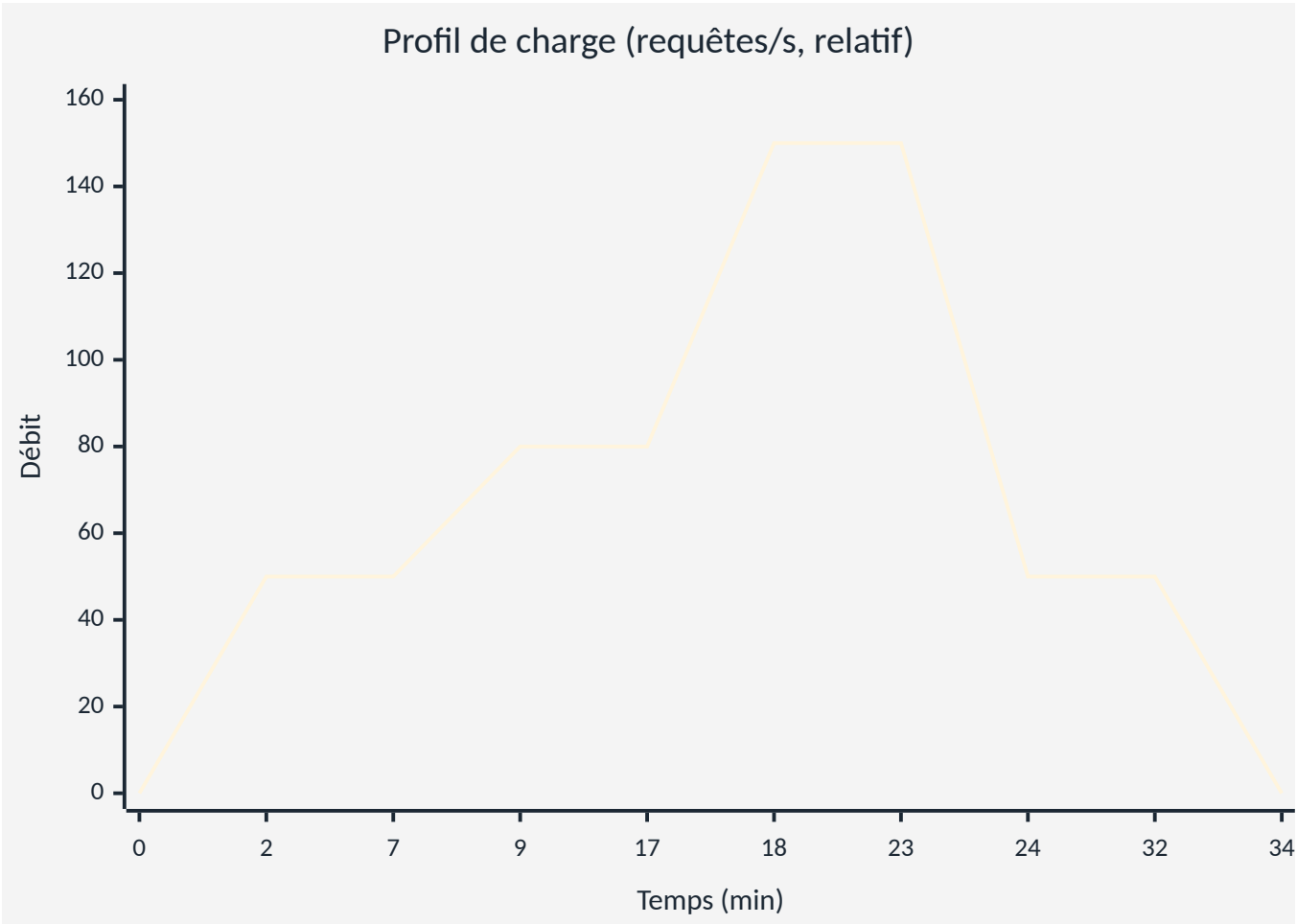
1. **Capacité minimale (C_min).** HPA désactivé, replicas au minimum, débit augmenté par paliers de 10 req/s toutes les 2 minutes. C_min est le débit maximal pour lequel p95 < 500 ms et erreurs < 1 %.
2. **Capacité maximale (C_max).** Même mesure avec les replicas au **maximum** de l'HPA.

Les niveaux du scénario principal sont ensuite définis **par rapport à ces capacités** :

Niveau	Débit visé	Effet attendu
Normal	~50 % de C_min	Aucune mise à l'échelle
Élevé	~80 % de C_max	L'HPA ajoute des replicas
Surcharge	~150 % de C_max	Au-delà de ce que la plateforme peut absorber

Ainsi, le protocole reste valable quelle que soit la puissance de la machine, et les résultats sont interprétables.

9.5 Scénario principal : Normal → High → Overload → Recovery



Phase	Durée	Débit	Ce qu'on observe
Montée	2 min	0 → normal	Préchauffage (JVM, caches, connexions)
Normal	5 min	normal	Référence : latence, erreurs, CPU (H1)
Rampe	2 min	normal → élevé	Dépassement de la cible CPU de 70 %
High	8 min	élevé	Réaction de l'HPA, arrivée des nouveaux pods, répartition du trafic (H2)
Rampe	1 min	élevé → surcharge	-
Overload	5 min	surcharge	HPA au maximum, alertes CPU / latence / erreurs, dégradation contrôlée (H3)
Recovery	8 min	retour au niveau normal	Retour aux objectifs, puis réduction progressive des replicas (H4)
Descente	2 min	→ 0	-

Comportement attendu en surcharge (dégradation contrôlée) :

- **Filles d'attente bornées.** Le nombre de threads et la file d'attente de chaque service Spring Boot sont limités. Au-delà, le service répond immédiatement `503` au lieu d'accumuler des requêtes jusqu'à manquer de mémoire.
- **Délais maximums** (section 3.6) : aucune requête n'attend indéfiniment.
- **Circuit breakers** : si le ml-service sature, risk-service arrête de l'appeler temporairement, et les transactions partent en REVUE. Elles ne sont jamais approuvées sans modèle.

- **Résultat attendu** : le taux d'erreur **dépasse** 5 % pendant la surcharge. L'alerte doit se déclencher, c'est le but. Mais aucun pod ne plante, et tout revient à la normale seul (H4).

9.6 Expériences de panne (chaos)

Chaque expérience se déroule sous **charge normale constante**, pour mesurer l'impact sur un trafic réel.

#	Panne injectée	Commande	Hypothèse	Mesures clés
C1	Suppression d'un pod risk-service	<code>kubectl delete pod</code>	H5	Erreurs pendant l'incident, délai jusqu'au nouveau pod prêt
C2	Suppression d'un pod ml-service	<code>kubectl delete pod</code>	H5	Idem, et chargement du modèle depuis le cache
C3	Suppression d'un pod api-gateway	<code>kubectl delete pod</code>	H5	Erreurs vues par k6 (Traefik relance la requête sur l'autre pod)
C4	Panne totale du ml-service	<code>kubectl scale --replicas=0</code> puis retour	H8	Part des transactions en REVUE, état du circuit breaker, 0 approbation , délai de retour à la normale
C5	Arrêt d'un nœud (worker-2)	<code>docker stop fintech-worker2</code>	H6	Disponibilité externe, délai de détection, délai de remplacement des pods
C6	Suppression du pod RabbitMQ	<code>kubectl delete pod</code>	H7	<code>outbox_pending</code> qui monte puis redescend, nombre d'événements d'audit = nombre de transactions
C7	Suppression du pod PostgreSQL	<code>kubectl delete pod</code>	H9	Durée d'interruption, erreurs, nombre de transactions avant / après
C8	Panne de MLflow	<code>kubectl scale --replicas=0</code>	-	Aucun impact attendu sur les prédictions (cache du modèle)
C9	Déploiement défectueux	Image qui échoue sa readiness probe	H10	0 erreur utilisateur, rolling update bloqué, rollback automatique

Deux réglages indispensables, issus de ces expériences :

1. **Détection rapide d'un nœud en panne (C5)**. Par défaut, Kubernetes attend **5 minutes** avant de déplacer les pods d'un nœud injoignable. Pour les Deployments sans état, cette tolérance est réduite à **30 s** (`tolerationSeconds` sur `node.kubernetes.io/unreachable` et `not-ready`). Sans ce réglage, H6 échouerait.
2. **Relance par Traefik (C3)**. Un middleware `retry` (2 tentatives) relance une requête qui échoue parce que son pod vient de disparaître. C'est sans risque ici : `POST /api/v1/transactions` est **idempotent** grâce au `trans_num` (section 3.4), donc une requête relancée ne crée jamais de doublon.

9.7 Définitions des mesures

Des définitions précises et fixées à l'avance, pour que les chiffres soient comparables et défendables :

Mesure	Définition	Source
Débit	Requêtes réussies par seconde (moyenne de la phase)	k6
Latence	p50, p95 et p99 de bout en bout, vues par le client	k6 (et p95 serveur pour comparaison)
Taux d'erreur	Réponses 5xx + délais dépassés / total des requêtes	k6
CPU / mémoire	Utilisation moyenne et maximale par service, par rapport aux limites	cAdvisor
Replicas	Nombre de pods prêts par service au cours du temps	kube-state-metrics

Mesure	Définition	Source
Délai de réaction de l'HPA	Du premier dépassement de 70 % de CPU jusqu'à l'augmentation des replicas souhaités	kube-state-metrics + événements Kubernetes
Délai de démarrage d'un pod	De la création du pod jusqu'à l'état prêt (readiness)	Événements Kubernetes
Temps de mise à l'échelle	Somme des deux précédents : du dépassement du seuil jusqu'aux nouveaux pods prêts	Calcul
Temps de récupération (charge)	Du retour au débit normal jusqu'à p95 < 500 ms et erreurs < 1 % pendant 1 min	k6
Temps de récupération (panne)	De l'injection de la panne jusqu'au retour à l'état nominal (replicas prêts, erreurs < 1 %)	Script de chaos + Prometheus
Impact utilisateur	Nombre et part des requêtes en erreur pendant la fenêtre de l'incident	k6
Perte de données	Nombre de transactions et d'événements d'audit avant et après, comparés à ce que k6 a envoyé	Requêtes SQL + k6

Précision des mesures. Pour les délais, les **horodatages des événements Kubernetes** (à la seconde) sont préférés aux métriques Prometheus, qui dépendent de l'intervalle de collecte.

9.8 Protocole d'exécution

Avant chaque expérience :

- cluster stable depuis au moins 10 min, avec tous les replicas au minimum ;
- simulateur arrêté : seul k6 génère du trafic ;
- aucun Job du pipeline ML en cours ;
- IDE et navigateur fermés, sauf Grafana pour l'observation ;
- version de chaque service et du modèle notée.

Déroulement automatisé (`make experiment NAME=C4 RUN=2`) :

1. Annotation « début » dans Grafana.
2. Lancement de k6, avec le profil de l'expérience.
3. Injection de la panne à l'instant prévu (par exemple T + 3 min).
4. Rétablissement si nécessaire (par exemple `scale` de retour).
5. Fin de k6 et annotation « fin ».
6. Export des résultats dans `load-tests/results/<date>-<expérience>-<run>/` : résumé JSON de k6, événements Kubernetes, requêtes de comptage SQL, métriques clés extraites de Prometheus.

Répétitions (décision D29). Chaque expérience est **exécutée 3 fois**. Le README rapporte la **médiane** et l'**étendue** (min-max) de chaque mesure. Une seule exécution pourrait être un hasard favorable.

Enregistrement. Le premier passage réussi de chaque expérience clé est **filmé** (écran Grafana + terminal) pour la démo. On évite ainsi de rejouer des expériences en direct le jour de la présentation (correctif de la revue).

9.9 Présentation des résultats

Pour chaque expérience, le README contient :

- l'**hypothèse** et le critère de réussite ;
- un **tableau des mesures** (médiane et étendue sur 3 exécutions) ;

- la **capture Grafana annotée** : phases et panne visibles ;
- un **verdict** : hypothèse validée ou non ;
- une **explication**, surtout en cas d'échec, avec la correction apportée si possible.

Exemple de tableau (C4, panne du ml-service) :

Mesure	Médiane	Min-Max	Critère	Verdict
Transactions approuvées sans modèle	...	...	0	...
Part des transactions en REVUE pendant la panne	...	...	100 %	...
Délai d'ouverture du circuit breaker	...	...	-	...
Délai de retour à la normale après rétablissement	...	...	< 60 s	...

9.10 Limites documentées

- **Réseau simulé.** kind ne reproduit pas la latence ni les pannes d'un vrai réseau entre les nœuds du cluster.
- **Réseau local entre k6 et le cluster.** Le Wi-Fi ajoute une variabilité de latence. Elle est limitée par un câble Ethernet et mesurée à chaque session.
- **Composants avec état non redondants.** PostgreSQL et RabbitMQ (section 6.4) : C6 et C7 mesurent le temps de récupération, mais ne peuvent pas démontrer une continuité totale.
- **Pas d'autoscaling des nœuds.** Un cluster local a un nombre fixe de nœuds : la mise à l'échelle se limite aux pods.

9.11 Livrables et calendrier

Livrable	Responsable	Semaine
Scripts k6, scripts de chaos, <code>make experiment</code> , annotations Grafana	P4	S9-S10
Test de capacité (C_min, C_max) et calibration des niveaux	P4	S10
Réglages : files bornées, tolérance des nœuds, relance Traefik	P1, P4	S10
Scénario principal × 3	P4, P5	S11
Expériences C1 à C9 × 3	P4, P5	S11-S12
Analyse, tableaux, captures, vidéos pour la démo	P4, P5	S12

10. Répartition des rôles et planning (version corrigée)

Cette section reprend le document de répartition initial de l'équipe. Elle intègre les correctifs de la revue et toutes les décisions prises pendant la conception (D1 à D32).

Dates clés :

- **Début** : lundi 28/09/2026 (S1).
- **Gel du code** : lundi 21/12/2026 (S13).
- **Dépôt visé** : samedi 09/01/2027.
- **Date limite officielle** : dimanche 10/01/2027.

10.1 Rôles

Membre	Rôle	Tâches du sujet	Périmètre principal
P1	Architecte backend et chef de projet	T1, intégration T6, décision T4	Les 7 services Spring Boot (Gateway, Auth, Customer, Transaction, Risk, Notification , Audit , décision D16), outbox, topologie RabbitMQ, contrats OpenAPI, tests de bout en bout, coordination, consolidation du README
P2	Data Scientist	T2, T3, T4	Données Sparkov, benchmark des 5 modèles, validation statistique, calibration et prédiction conforme, choix de α , audit d'équité
P3	Ingénieur MLOps et serving	T5, T6, T7	MLflow, pipeline automatisé avec portes de qualité, ml-service FastAPI, visualisations, simulateur (rejeu et rejets de paiement), suppléant de P1
P4	Ingénieur DevOps	T8, T9, T11	Images Docker, cluster kind, manifests, Argo CD, Sealed Secrets, CI/CD et DevSecOps, tests de charge et de panne
P5	Frontend et observabilité	Intégration T7, T10, interface T4	Application React, Prometheus, Grafana, Loki, alertes Discord, runbooks, script et enregistrement de la démo

Changements par rapport au document initial :

Changement	Raison
Notification et Audit passent de P4 à P1	Décision de l'équipe (D16) : tout le code Spring Boot sous une même responsabilité
P3 devient suppléant officiel de P1	P1 porte désormais la charge la plus lourde de l'équipe. P3 a du temps disponible en S1-S5 et peut reprendre un service (par exemple customer-service) si P1 prend du retard
Le simulateur est confié à P3	Il rejoue les données Sparkov et envoie les rejets de paiement simulés (D19), ce qui est proche du travail de P3
Les contrôles de sécurité de la CI sont avancés à S4-S6	Correctif 3 de la revue : lisser la charge de P4
Un premier vrai modèle est en production dès S8	Correctif 4 de la revue : les tests de charge ne dépendent plus du modèle final
Le jeu de données est choisi en S1	Correctif 2 de la revue : les contrats OpenAPI en dépendent

10.2 Couverture des tâches du sujet

Tâche	Responsable	Support	Section
T1 Architecture microservices	P1	P3 (suppléant)	3
T2 Benchmark ML	P2	P3 (MLflow)	4

Tâche	Responsable	Support	Section
T3 Validation statistique	P2	P3 (intégration au pipeline)	4
T4 Quantification de l'incertitude	P2	P1 (politique de décision), P5 (file de revue dans React)	4, 2.5
T5 MLflow et MLOps	P3	P2 (code ML), P4 (image du pipeline)	5
T6 Serving FastAPI	P3	P1 (intégration risk-service)	5
T7 Visualisation	P3	P5 (pages React)	4.11, 5
T8 Docker et Kubernetes	P4	P1, P3 (probes, métriques)	6
T9 Charge et haute disponibilité	P4	P5 (observation), P1 (réglages)	9
T10 Supervision	P5	P1, P3 (métriques), P4 (installation)	8
T11 CI/CD et DevSecOps	P4	P1 (tests de bout en bout), P3 (CI du code ML)	7

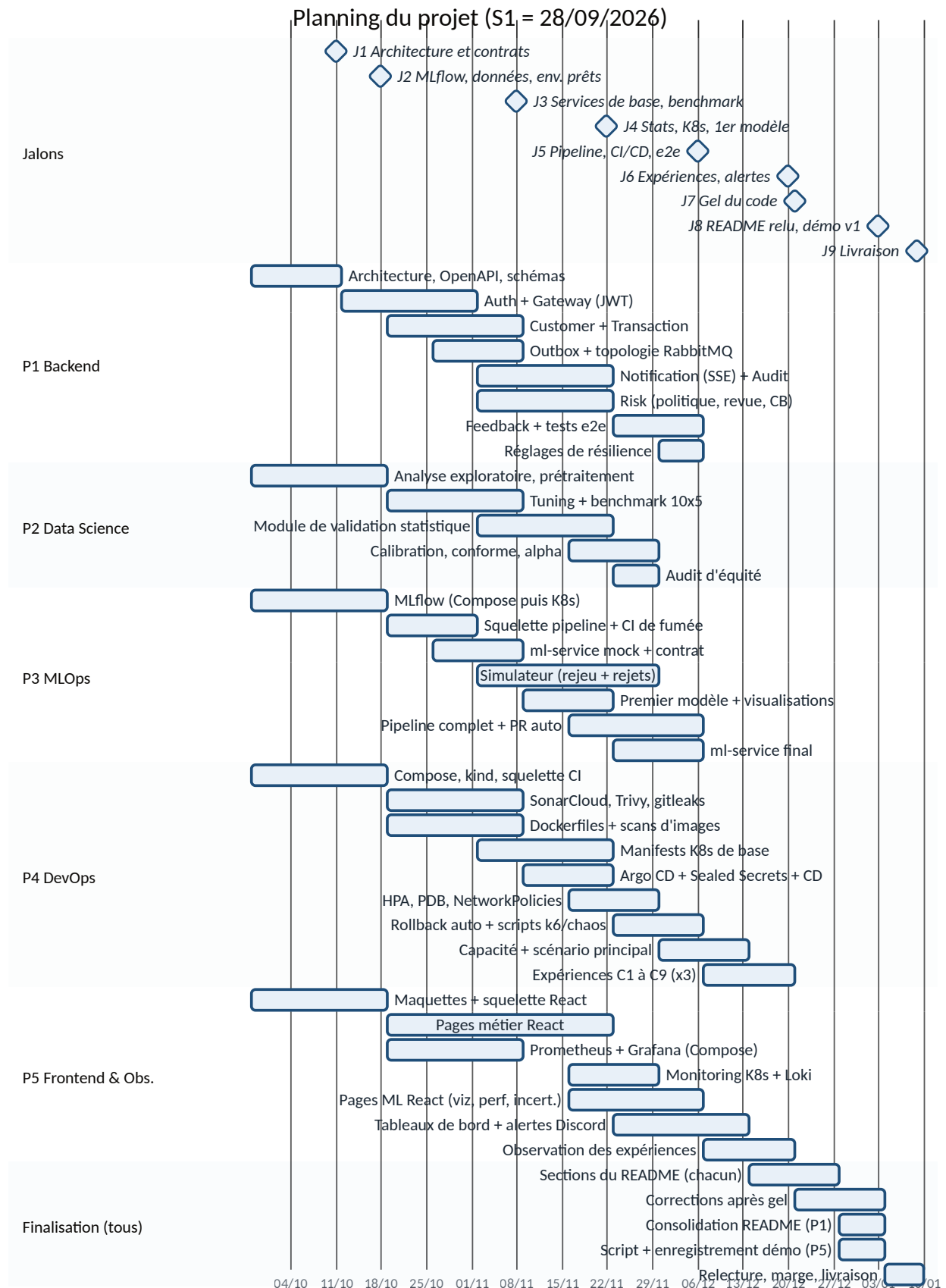
10.3 Charge de travail par période

Membre	S1-S3 (fondations)	S4-S8 (développement)	S9-S12 (industrialisation et expériences)	S13-S15 (finalisation)
P1	● Contrats, schémas, dépôt	● 7 services, outbox	● Feedback, tests e2e, réglages	● README consolidé
P2	● Analyse exploratoire, prétraitement	● Benchmark, statistiques	● Incertitude, α , équité	● Section README
P3	● MLflow (+ suppléance P1)	● Mock, simulateur, premier modèle	● Pipeline, portes, PR auto	● Section README
P4	● Compose, kind, CI	● Sécurité CI, images, manifests	● Argo CD, HPA, expériences	● Section README
P5	● Maquettes, squelette React	● Pages React, Prometheus local	● Tableaux de bord, alertes, Loki	● Démo

● charge forte · ● charge moyenne · ● charge légère

Chaque membre a au plus deux périodes de charge forte. **P1 reste le point de vigilance.** Son avancement est vérifié à chaque réunion hebdomadaire, et P3 intervient dès qu'un jalon de P1 glisse d'une semaine.

10.4 Planning (S1 à S15)



Période des fêtes (S13-S14). Aucune tâche de développement n'est planifiée après le gel du code. S13 et S14 sont réservées aux corrections, à la documentation et à la démo, qui peuvent se faire en travail allégé.

10.5 Chaîne des livrables transmis

#	Livrable	De	Vers	Fin
1	Choix du jeu de données (fait : Sparkov, D1)	Équipe	Tous	S1
2	Contrats OpenAPI, schémas de base, schéma d'architecture	P1	P3, P4, P5	S2
3	Serveur MLflow opérationnel + script de suivi type	P3	P2	S3
4	Données prétraitées, liste des features, format de transaction	P2	P1, P3	S3
5	Docker Compose, cluster kind, squelette CI	P4	Tous	S3
6	Contrat <code>/predict</code> + ml-service en mode mock	P3	P1	S6
7	Services de base + endpoints REST stables	P1	P4, P5	S6
8	Résultats du benchmark (meilleur modèle provisoire)	P2	P3	S6
9	Module de validation statistique	P2	P3	S7
10	Premier vrai modèle en production + visualisations	P3	P1, P5	S8
11	Cluster avec manifests, Argo CD, cibles de supervision	P4	P5	S8
12	Modèle final, α , seuils, rapport statistique	P2	P3, P1	S9
13	Pipeline complet + ml-service final	P3	P4	S10
14	Tests e2e intégrés au déploiement	P1	P4	S10
15	Tableaux de bord et alertes	P5	P4	S11
16	Résultats des expériences + vidéos	P4, P5	Tous	S12
17	README consolidé + démo	P1, P5	Tous (relecture)	S14

Règle de passage (conservée du document initial). Un livrable n'est considéré comme transmis que s'il est **documenté, testé et démontré** à l'équipe.

10.6 Jalons de contrôle

Jalon	Date	Critère de validation
J1	11/10/2026	Architecture validée (ce document), dépôt et ruleset en place, contrats OpenAPI v1
J2	18/10/2026	MLflow prêt, données prétraitées, Docker Compose et CI opérationnels
J3	08/11/2026	Services de base fonctionnels, benchmark terminé, ml-service mock livré, contrôles de sécurité actifs en CI
J4	22/11/2026	Validation statistique terminée, plateforme déployée par Argo CD, premier vrai modèle en production , Prometheus installé
J5	06/12/2026	Modèle final, pipeline avec portes de qualité, CI/CD complète, tests e2e et rollback automatique fonctionnels
J6	20/12/2026	Toutes les expériences réalisées 3 fois, tableaux de bord et alertes validés, vidéos enregistrées
J7	21/12/2026	Gel du code : plus de nouvelle fonctionnalité, uniquement des corrections
J8	03/01/2027	README complet relu par tous, démo enregistrée en première version
J9	09/01/2027	Livraison : dépôt GitHub, README, démo de 5 min maximum (un jour de marge avant le 10/01)

10.7 Organisation de l'équipe

Pratique	Mise en œuvre
Suivi des tâches	GitHub Projects : une carte par tâche, liée à ses PR
Réunion hebdomadaire	30 min chaque dimanche : avancement, blocages, jalon de la semaine
Point de synchronisation	Message court sur Discord chaque jour : fait, à faire, blocage
Revue de code	Chaque PR relue par un autre membre, qui est aussi celui qui reçoit le livrable (CODEOWNERS)
Décisions	Toute nouvelle décision d'architecture est ajoutée au tableau de la section 2.6

10.8 Risques et mesures

Risque	Probabilité	Impact	Mesure
Surcharge de P1 (7 services + coordination)	Élevée	Élevé	P3 suppléant officiel, suivi hebdomadaire, services simples (Notification, Audit) livrables en version minimale
Mémoire insuffisante (16 Go)	Moyenne	Élevé	Budget mesuré en S8, maximums HPA réduits si besoin, k6 sur un autre portable (D28), démo sur la machine la plus puissante de l'équipe
Bot CI bloqué par la protection de <code>main</code>	Moyenne	Moyen	Mécanisme validé par P4 dès S2 (section 7.6)
NetworkPolicies non appliquées par kindnet	Faible	Moyen	Test en S2, passage à Calico sinon (section 6.2)
Retard du modèle final	Moyenne	Moyen	Premier vrai modèle en production dès S8, les expériences n'en dépendent pas
Calculs ML trop longs (SVM, MLP, 50 entraînements chacun)	Moyenne	Moyen	Sous-échantillonnage (4.4), approximation de noyau pour le SVM, exécution de nuit si nécessaire
Panne ou perte de la machine du cluster	Faible	Élevé	Tout est dans Git : <code>make cluster</code> recrée l'environnement sur une autre machine en moins d'une heure
Réseau local instable pendant les expériences	Moyenne	Moyen	Câble Ethernet, mesure de la latence de base à chaque session
Période des fêtes (S13-S14)	Certaine	Moyen	Développement terminé en S12, gel du code en S13, un jour de marge avant la date limite
Démo trop longue (> 5 min)	Moyenne	Élevé	Script minuté par P5, séquences vidéo des expériences enregistrées en S12, répétition complète en S14